Lecture 6:

Clustering and Segmentation

Harbin Institute of Technology (Shenzhen)

Jingyong Su

sujingyong@hit.edu.cn

Outline

- Introduction

- Clustering
  - ✓ Agglomerative
  - ✓ K-Means
  - ✓ Mean-Shift

Image Segmentation

- Goal: identify groups of pixels that go together

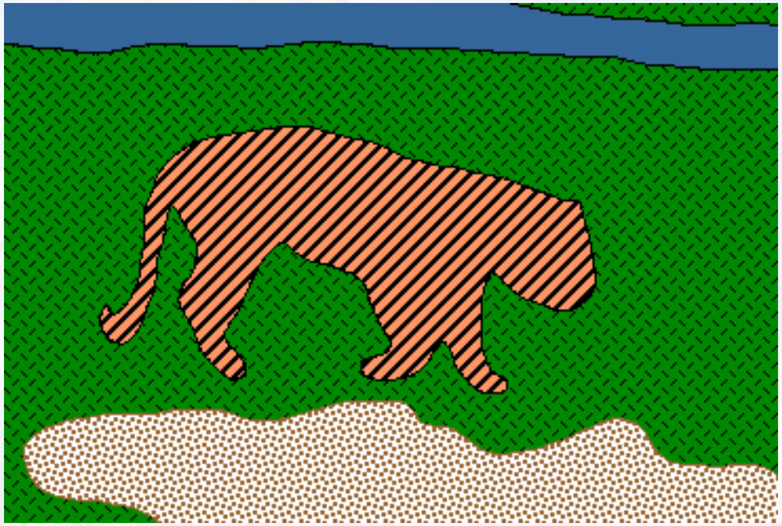
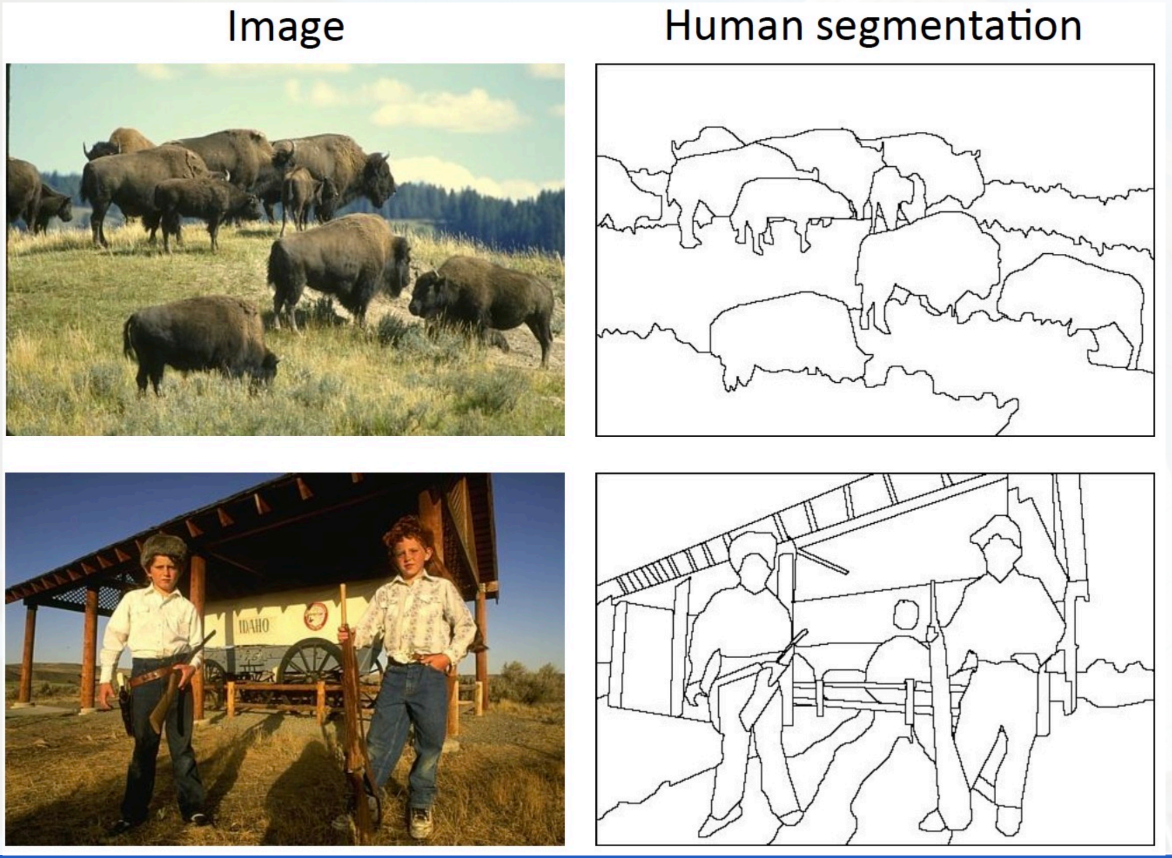


Image Segmentation

- Separate image into coherent "objects"



The Goals of Segmentation

- Separate image into coherent “objects”
- Group together similar-looking pixels for efficiency of further processing

“Superpixels”

Segmentation as a Result

Segmentation as Clustering

- Clustering: group together similar points and represent them with a single token

Key challenges:

(1) What makes two points/patches/images similar?
(2) How do we compute an overall grouping from pairwise similarities?

How Do We Cluster?

- Agglomerative clustering
  – Start with each point as its own cluster and iteratively merge the closest clusters

- K-means
  – Iteratively re-assign points to the nearest cluster center

- Mean-shift clustering
  – Estimate modes of pdf

General Ideas

- Tokens
  - whatever we need to group (pixels, points, surface elements, etc.)
- Bottom up clustering
  - tokens belong together because they are locally coherent
- Top down clustering
  - tokens belong together because they lie on the same visual entity (object, scene…)

>These two are not mutually exclusive

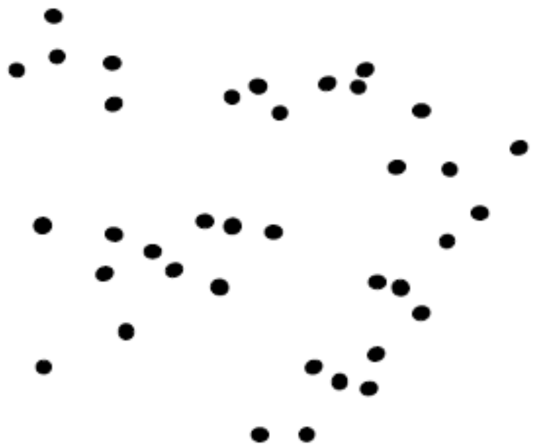
Outline

- Introduction
- Clustering
  - ✓ Agglomerative
  - ✓ K-Means
  - ✓ Mean-Shift

Agglomerative Clustering

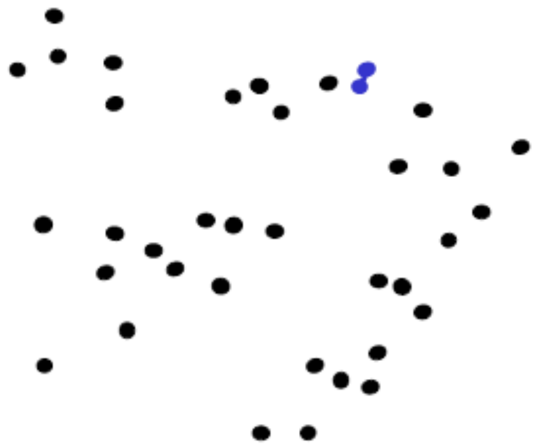
- Clustering is an unsupervised learning method.
- Given $x_1, \ldots, x_n \in \mathbb{R}^D$, the goal is to group them into clusters.
- The general idea behind agglomerative clustering is to look at similarities between points to decide how these points should be grouped in a sensible manner.
- We need a pairwise distance/similarity function.
  - Euclidean distance
  - Cosine similarity
  - Etc.

Agglomerative Clustering



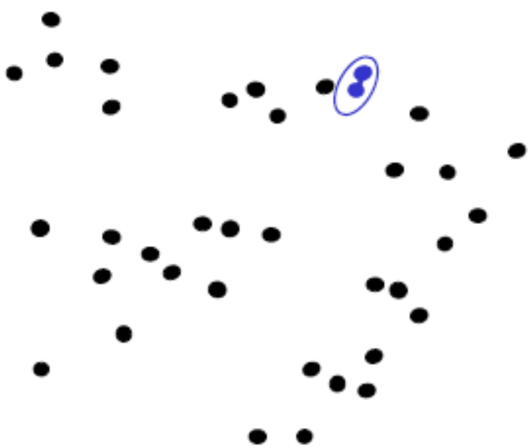
1. Say "Every point is its own cluster"

Agglomerative Clustering



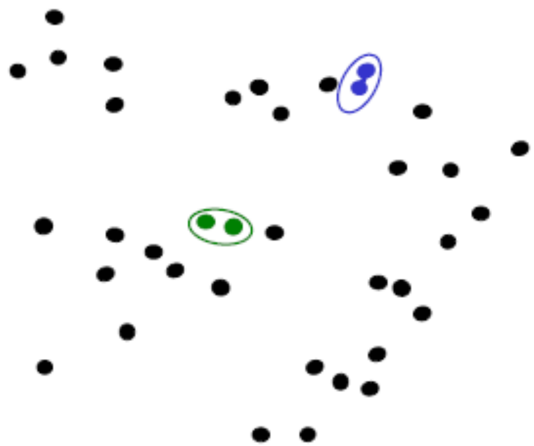
1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters

Agglomerative Clustering



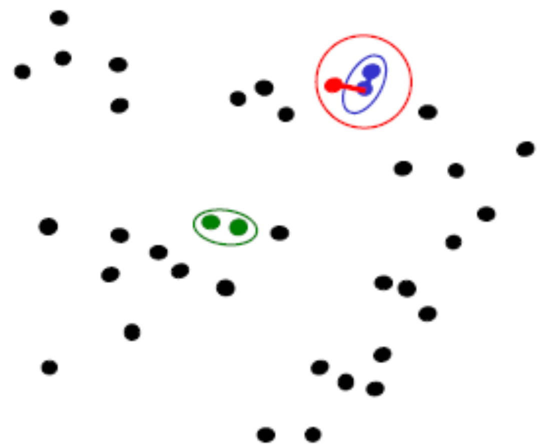
1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent cluster

Agglomerative Clustering



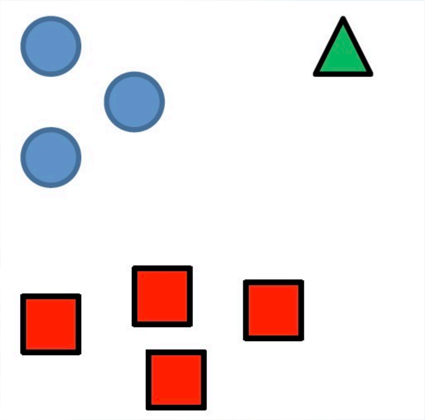
1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent cluster
4. Repeat

Agglomerative Clustering



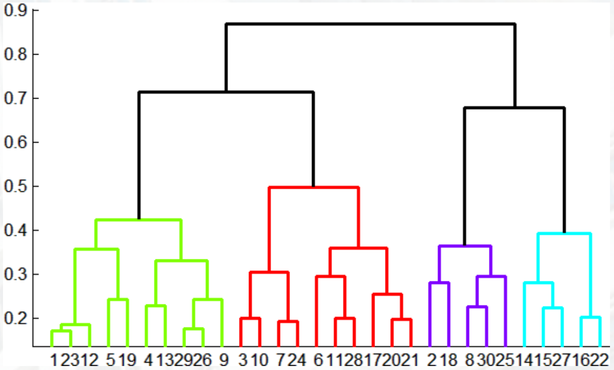
1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent cluster
4. Repeat

Agglomerative Clustering



How to define cluster similarity?
- Average distance between points,
- Maximum distance (complete-linkage)
- Minimum distance (single-linkage)
- Distance between means or medoids

How many clusters?
- Clustering creates a dendrogram (a tree)
- Threshold based on max number of clusters or based on distance between merges



Agglomerative Clustering

Algorithm

1. Initialize each point as its own cluster

2. Find the most similar pair of clusters

3. Merge the similar pair of clusters into a parent cluster

4. Repeat steps 2 & 3 until we have only 1 cluster.

Agglomerative Clustering

Good

- Simple to implement, widespread applications
- Clusters have adaptive shapes
- Provides a hierarchy of clusters

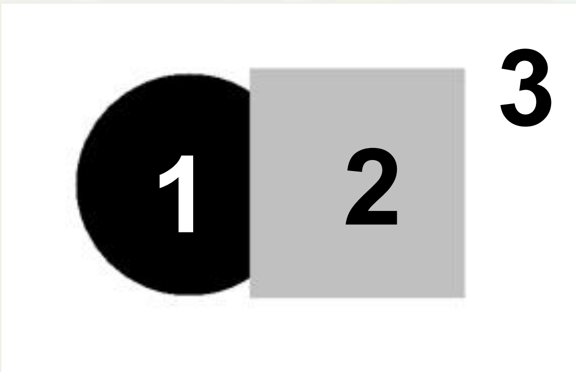
Bad

- May have imbalanced clusters
- Still have to choose number of clusters or threshold
- Need to use an "ultrametric" to get a meaningful hierarchy

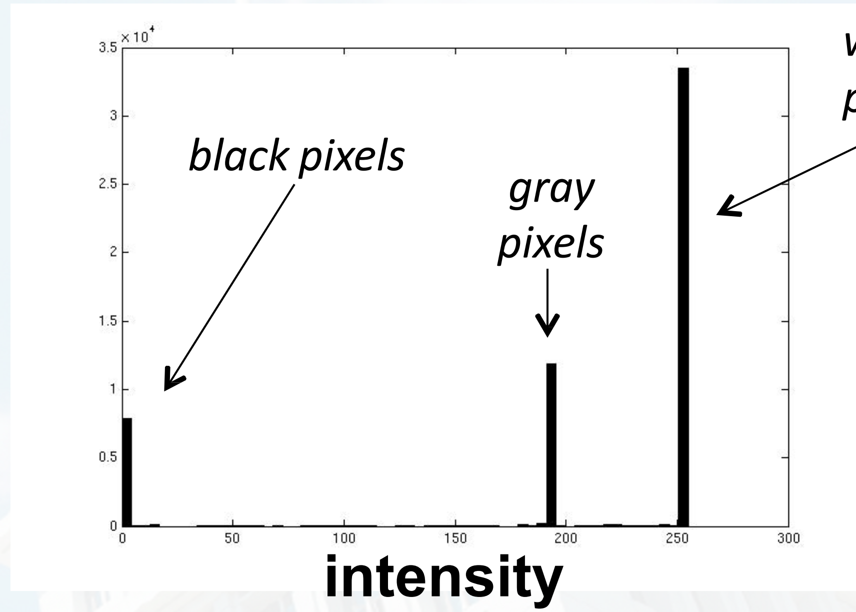
Outline

- Introduction
- Clustering
  - ✓ Agglomerative
  - ✓ K-Means
  - ✓ Mean-Shift

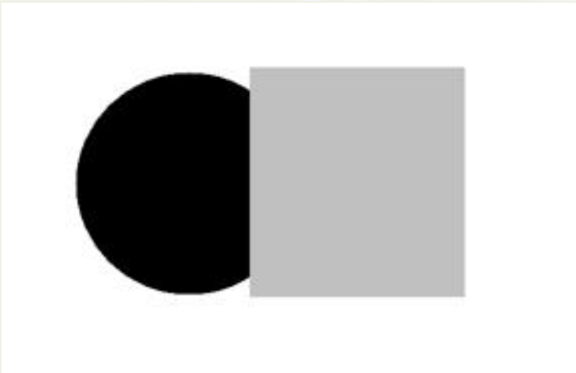
Image Segmentation: Toy Example



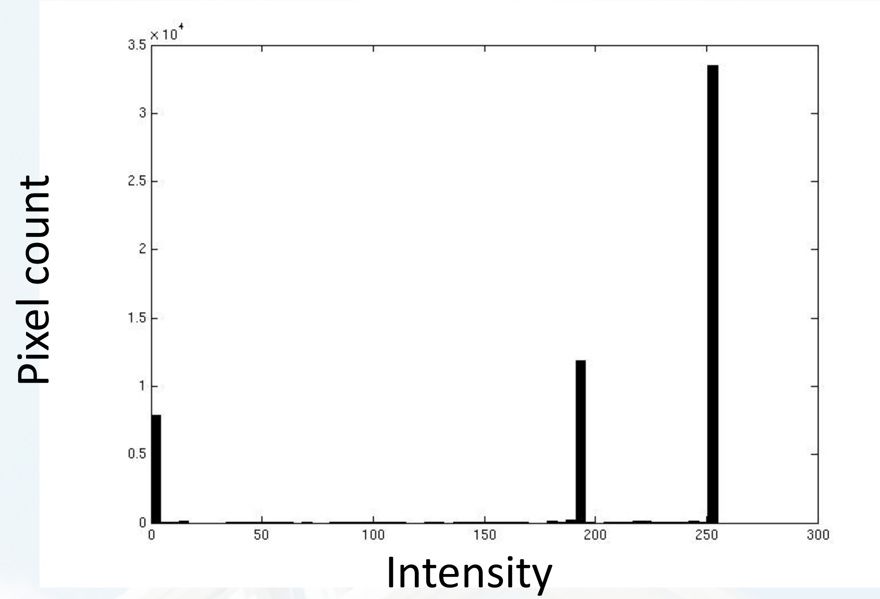
input image

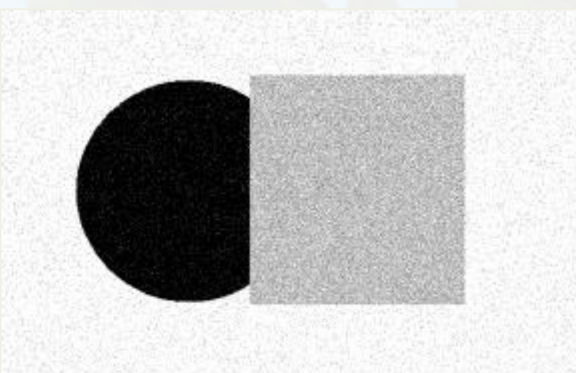


- These intensities define the three groups.
- We could label every pixel in the image according to which of these primary intensities it is.
  - i.e., segment the image based on the intensity feature.
- What if the image isn’t quite so simple?

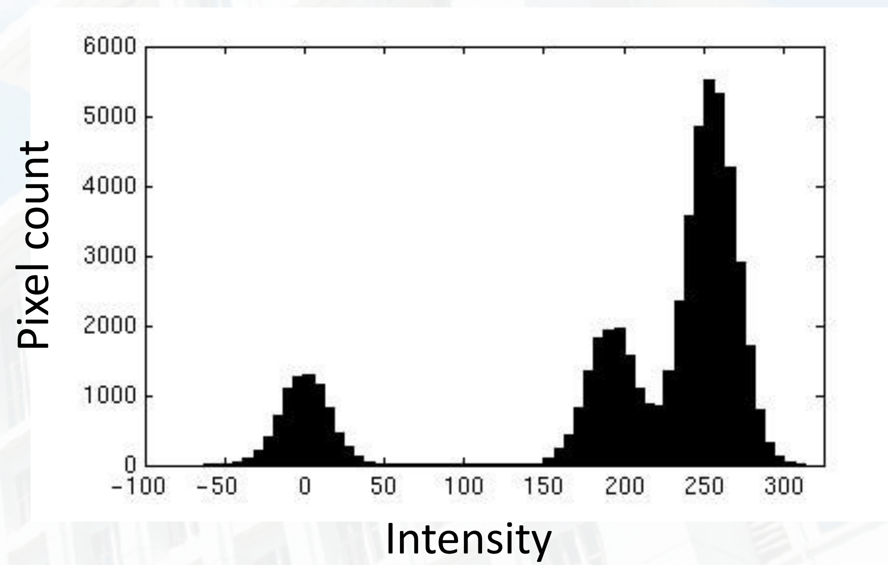


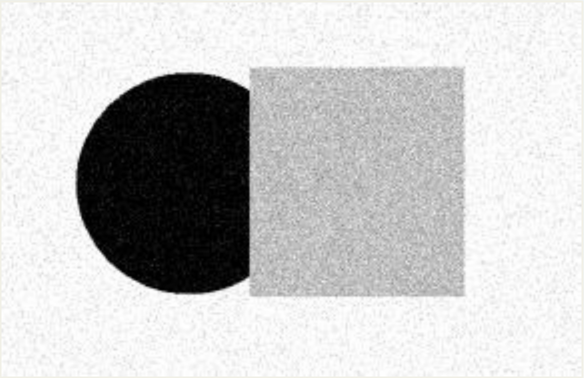
Input image



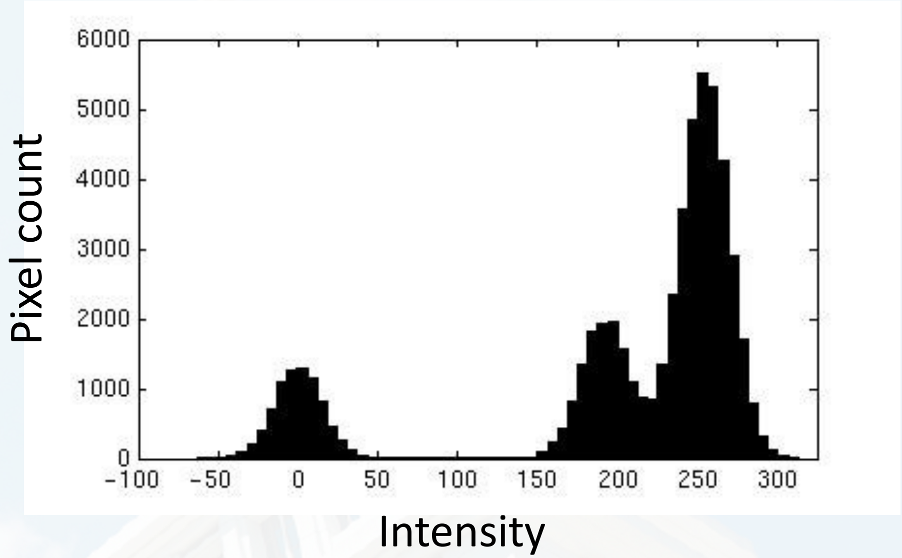


Input image

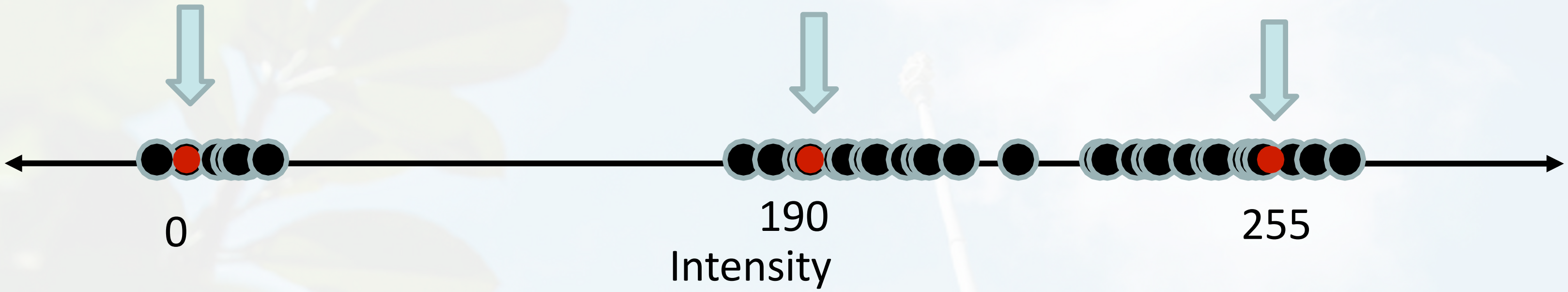


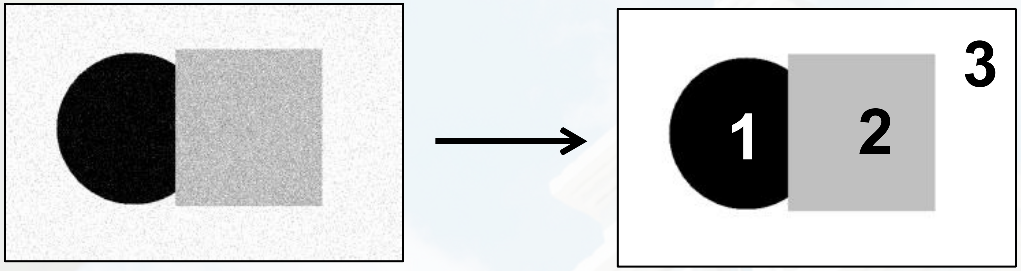


Input image



- Now how to determine the three main intensities that define our groups?
- We need to cluster.





- Goal: choose three “centers” as the representative intensities and label every pixel according to which of these centers it is nearest to.
- Best cluster centers are those that minimize Sum of Square Distance (SSD) between all points and their nearest cluster center c_i:

$$SSD = \sum_{Cluster\ i} \sum_{x \in cluster\ i} (x - c_i)^2$$

Clustering for Summarization

Goal: cluster to minimize variance in data given clusters

– Preserve information

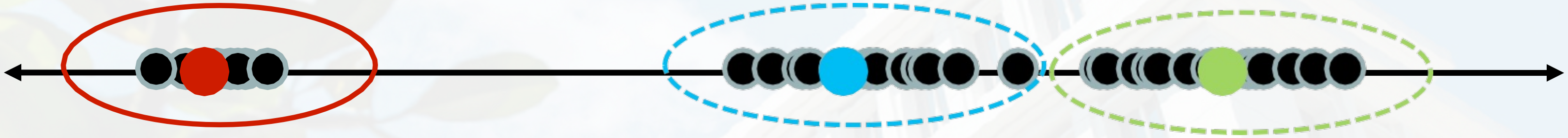
Cluster center Data

$$c^*,\ \delta^* = \underset{c,\,\delta}{\operatorname{argmin}} \frac{1}{N} \sum_{j}^{N} \sum_{i}^{K} \delta_{ij} \left(c_i - x_j \right)^2$$

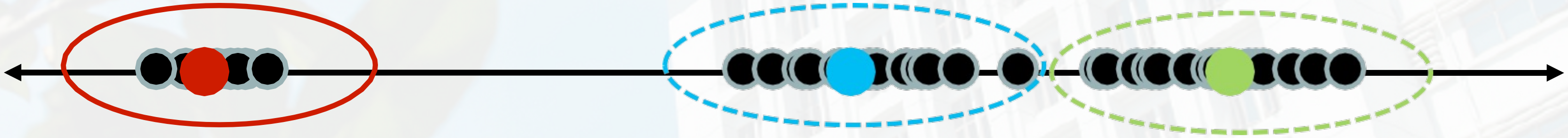
Whether x_j is assigned to c_i

Clustering

- With this objective, it is a "chicken and egg" problem:
  - If we knew the *cluster centers*, we could allocate points to groups by assigning each to its closest center.



  - If we knew the *group memberships*, we could get the centers by computing the mean per group.



K-Means Clustering

1. Initialize ($t = 0$): cluster centers $c_1, \ldots, c_K$

2. Compute δ^t: assign each point to the closest center
   – δ^t denotes the set of assignment for each x_j to cluster c_i at iteration *t*

$$\delta^t = \operatorname*{argmin}_{\delta} \frac{1}{N} \sum_{j}^{N} \sum_{i}^{K} \delta_{ij}^{t-1} \left(c_i^{t-1} - x_j\right)^2$$

3. Computer c^t: update cluster centers as the mean of the points

$$c^t = \operatorname*{argmin}_{c} \frac{1}{N} \sum_{j}^{N} \sum_{i}^{K} \delta_{ij}^{t} \left(c_i^{t-1} - x_j\right)^2$$

4. Update $t = t + 1$, Repeat Step 2–3 till stopped

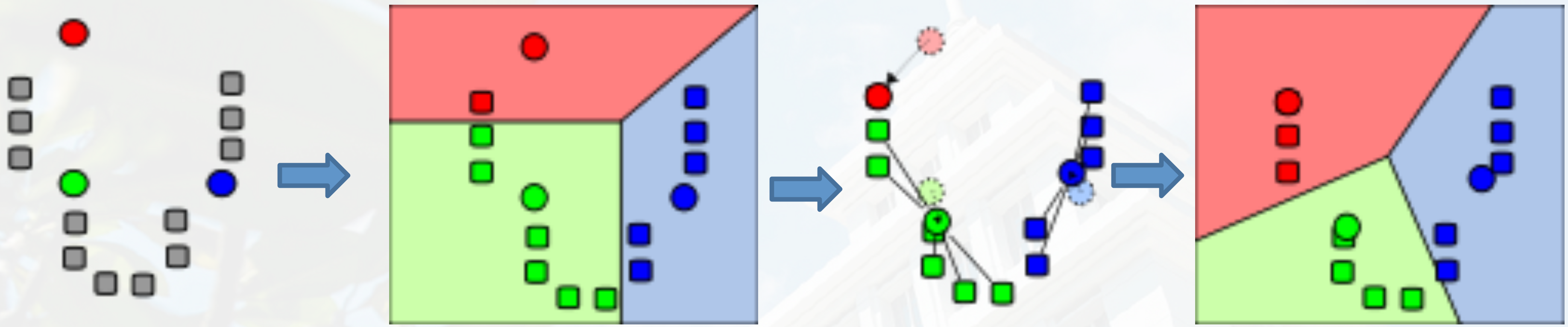
K-Means Clustering

1. Initialize ($t = 0$): cluster centers $c_1,\ldots,c_K$
   - Commonly used: random initialization
   - Or greedily choose K to minimize residual
2. Compute δ^t: assign each point to the closest center
   - Typical distance measure:
     - Euclidean
     - Cosine
     - Others
3. Computer c^t: update cluster centers as the mean of the points

$$c^t = \underset{c}{\operatorname{argmin}} \frac{1}{N}\sum_{j}^{N}\sum_{i}^{K}\delta_{ij}^{t}\left(c_i^{t-1} - x_j\right)^2$$

4. Update $t = t + 1$, Repeat Step 2–3 till stopped
   - c^t doesn't change anymore.

K-Means Clustering



1. Initialize Cluster Centers

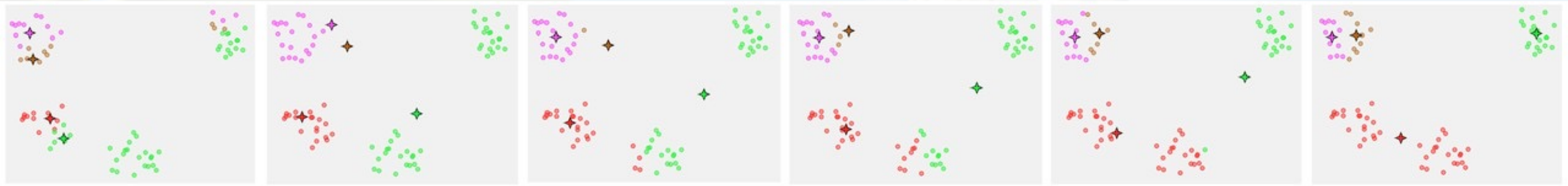
2. Assign Points to Clusters

3. Re-compute Means

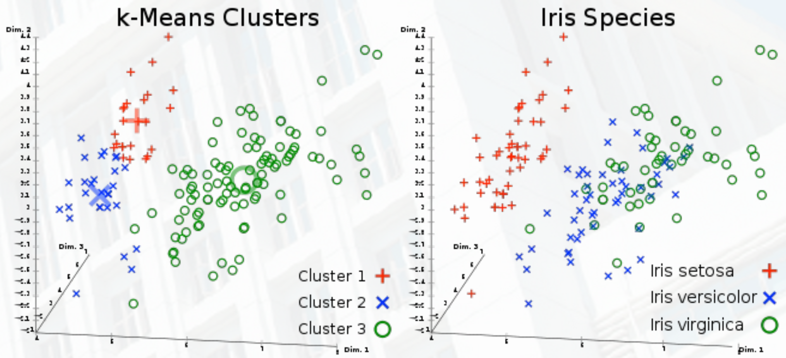
Repeat (2) and (3)

K-Means Clustering

- Converges to a *local minimum* solution
  - Initialize multiple runs



- Better fit for spherical data

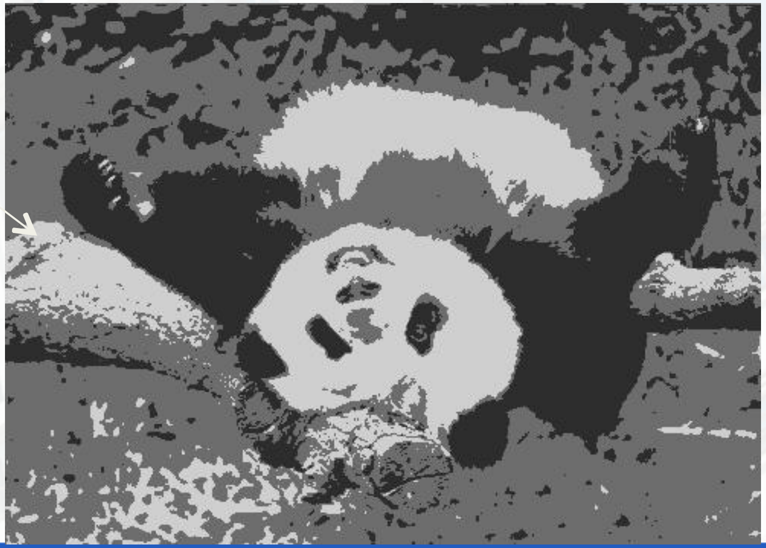


- Need to pick K (# of clusters)

Segmentation as Clustering

K=2

K=3



Feature Space

- Depending on what we choose as the *feature space*, we can group pixels in different ways.

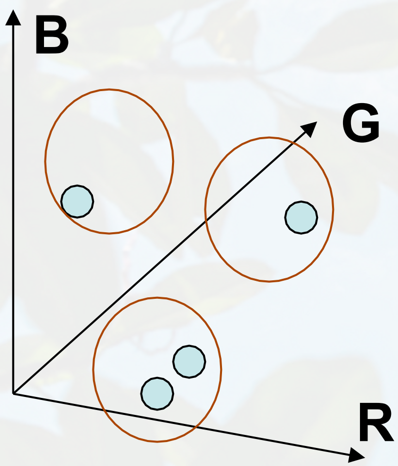
- Grouping pixels based on intensity similarity

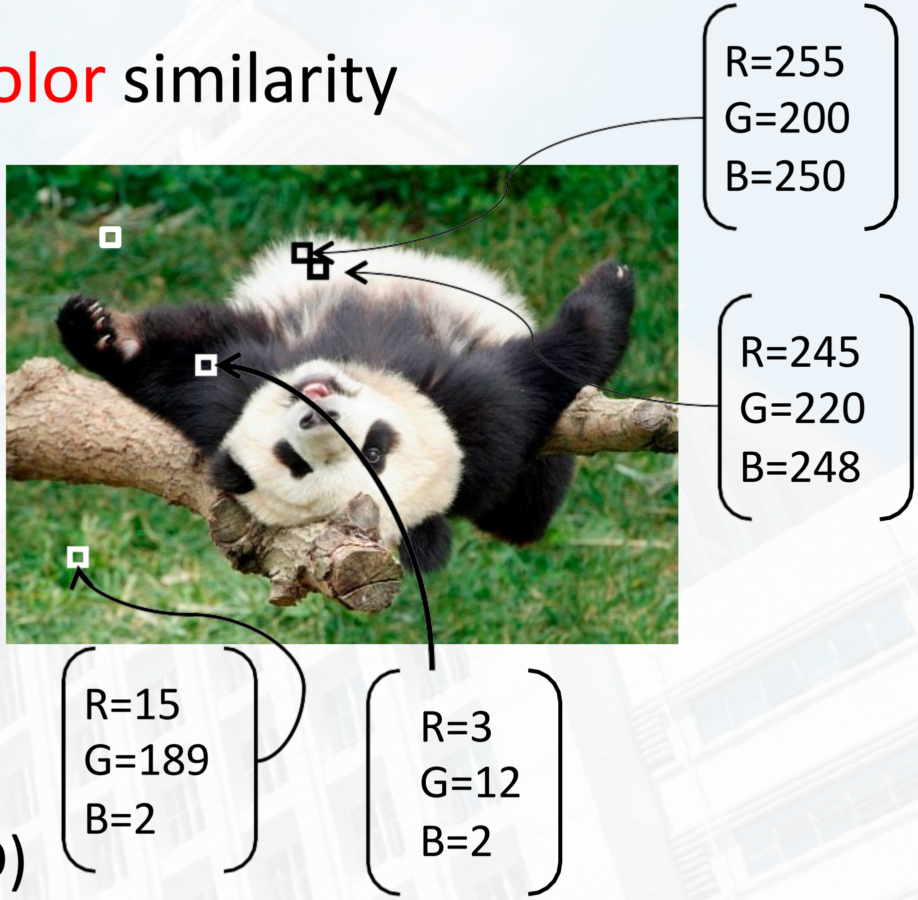
- Feature space: intensity value (1D)

Feature Space

- Depending on what we choose as the feature space, we can group pixels in different ways.

- Grouping pixels based on color similarity

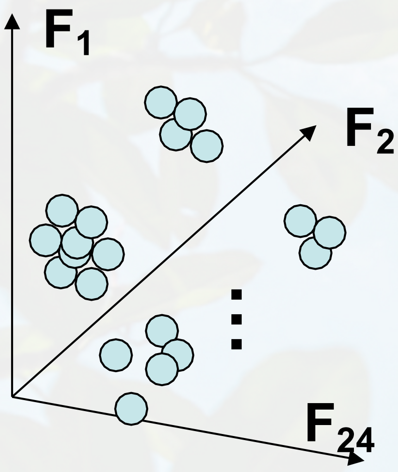


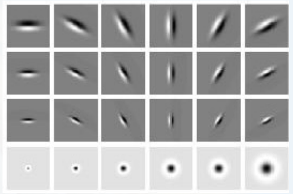


- Feature space: color value (3D)

Feature Space

- Depending on what we choose as the feature space, we can group pixels in different ways.
- Grouping pixels based on texture similarity



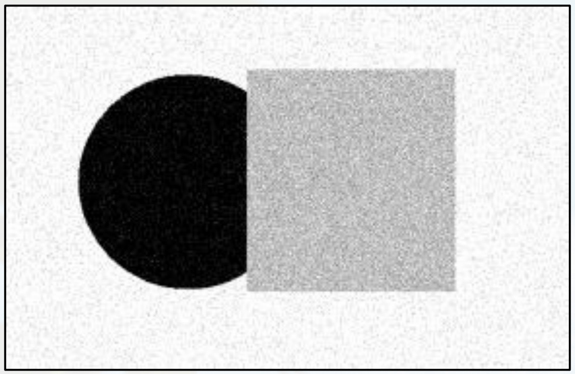


Filter bank of 24 filters

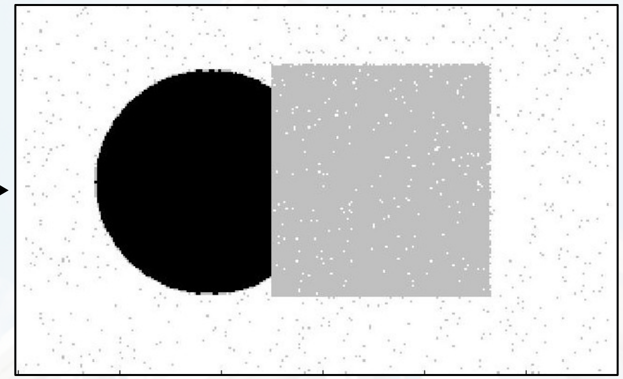
- Feature space: filter bank responses (e.g., 24D)

Smoothing Out Cluster Assignments

- Assigning a cluster label per pixel may yield outliers:

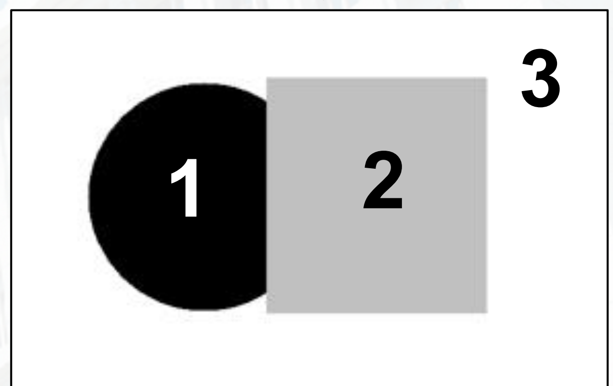


Original



Labeled by cluster center's intensity

?

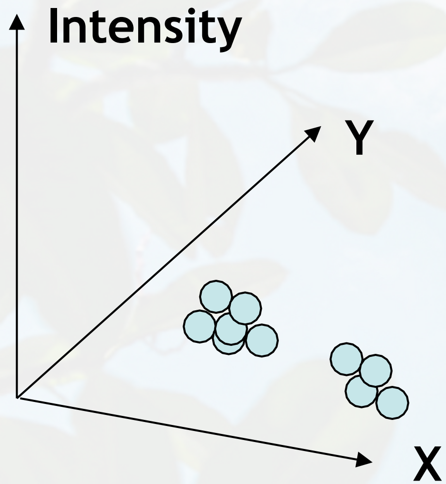


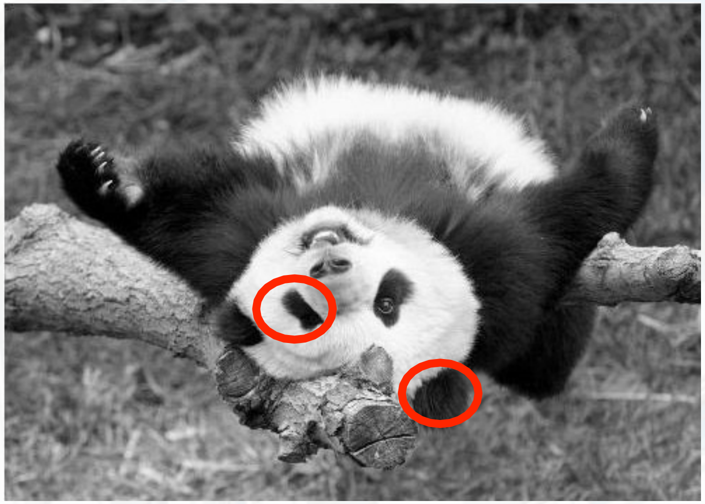
- How can we ensure they are spatially smooth?

Segmentation as Clustering

- Depending on what we choose as the *feature space*, we can group pixels in different ways.

- Grouping pixels based on *intensity+position* similarity





- Way to encode both *similarity* and *proximity.*

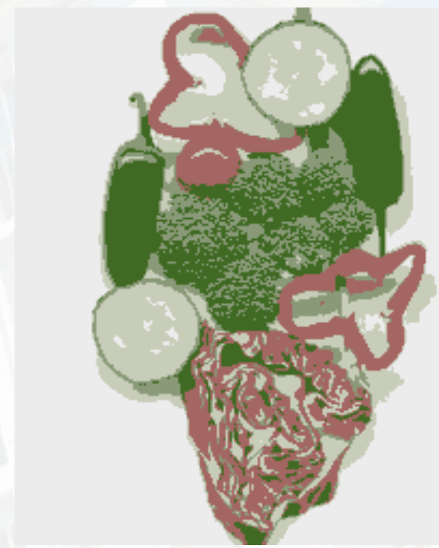
K-Means Clustering Results

- K-means clustering based on intensity or color is essentially vector quantization of the image amributes
  - Clusters don’t have to be spatially coherent

Image

Intensity--based clusters

Color--based clusters



K-Means Clustering Results

- K-means clustering based on intensity or color is essentially vector quantization of the image amributes
  - Clusters don’t have to be spatially coherent
- Clustering based on (r,g,b,x,y) values enforces more spatial coherence

How to evaluate clusters?

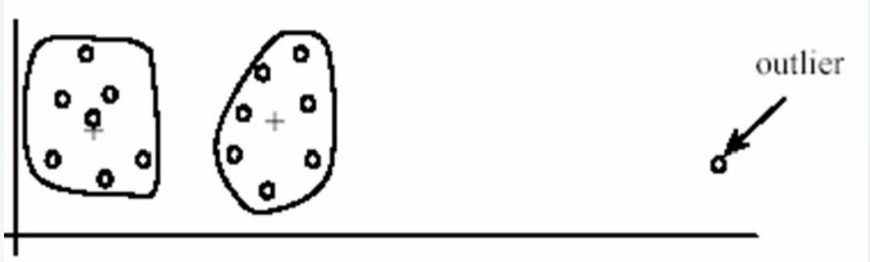
- Generative
  - How well are points reconstructed from the clusters?
- Discriminative
  - How well do the clusters correspond to labels?
    - Purity
  - Note: unsupervised clustering does not aim to be discriminative

How to choose the number of clusters?

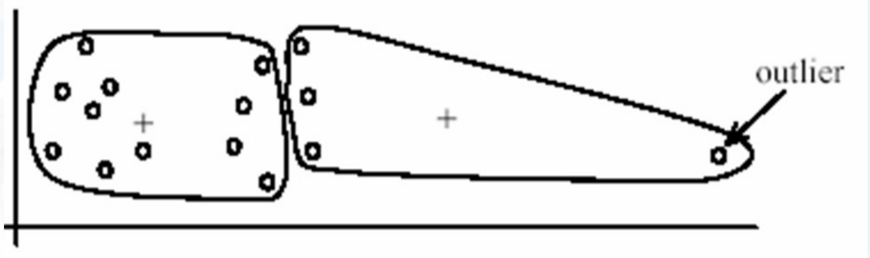
- Validation set
  - Try different numbers of clusters and look at performance
    - When building dictionaries (discussed later), more clusters typically work better

K-Means Pros and Cons

- Pros
  - Finds cluster centers that minimize conditional variance (good representation of data)
  - Simple and fast, Easy to implement
- Cons
  - Need to choose K
  - Sensitive to outliers
  - Prone to local minima
  - All clusters have the same parameters (e.g., distance measure is non– adaptive)
  - *Can be slow: each iteration is O(KNd) for N d--dimensional points
- Usage
  - Unsupervised clustering
  - Rarely used for pixel segmentation



(B): Ideal clusters



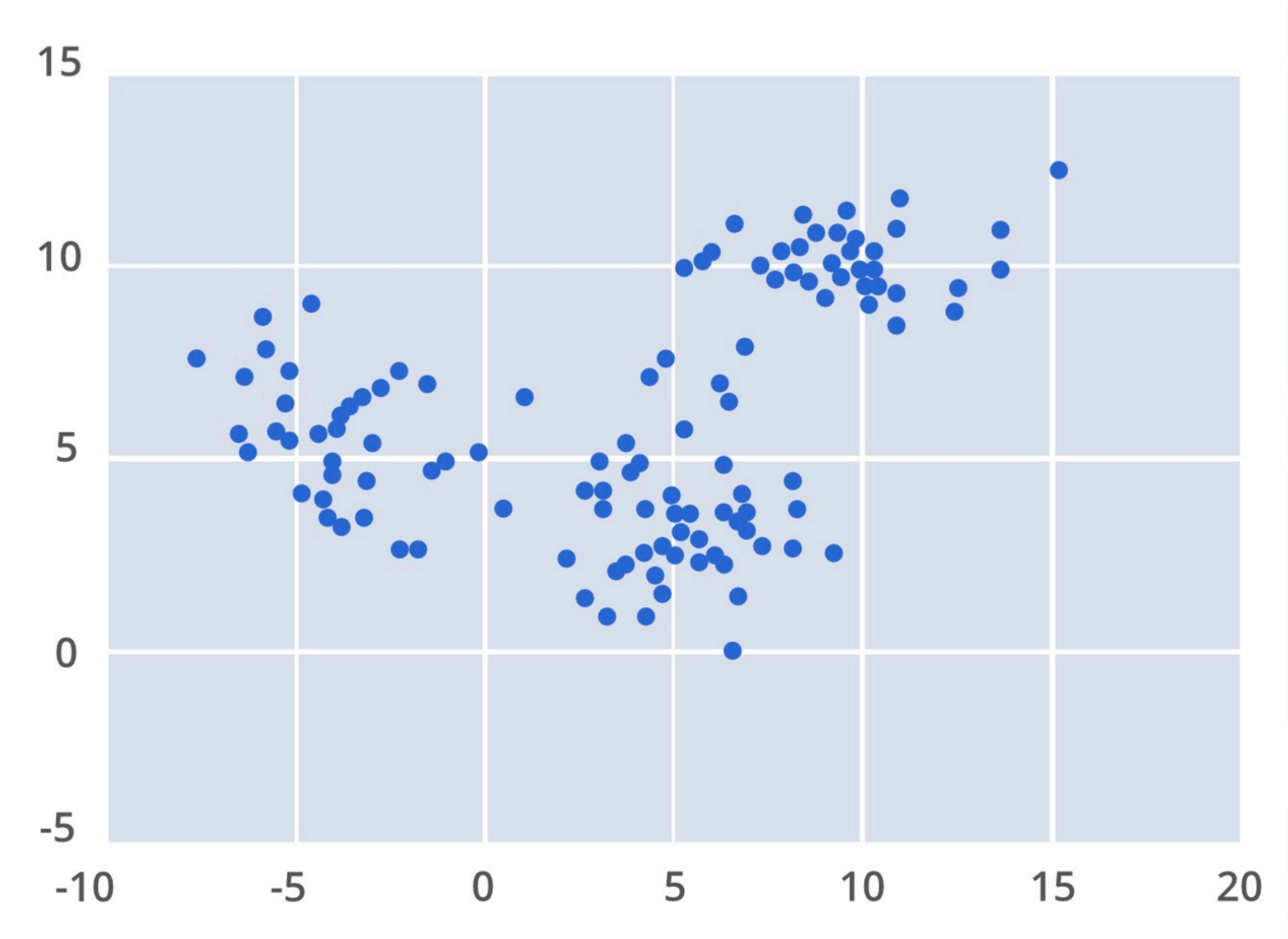
(A): Two natural clusters

(B): *k*-means clusters

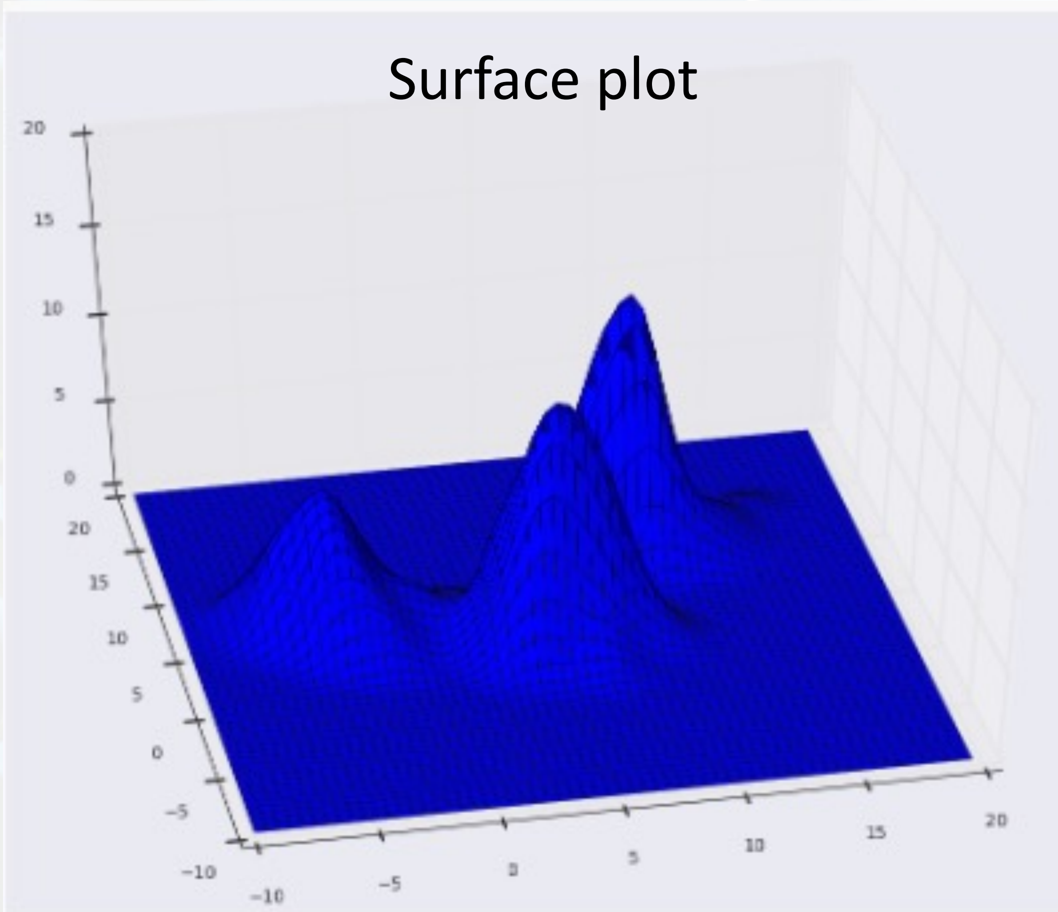
Outline

- Introduction
- Clustering
  - ✓ Agglomerative
  - ✓ K-Means
  - ✓ Mean-Shift

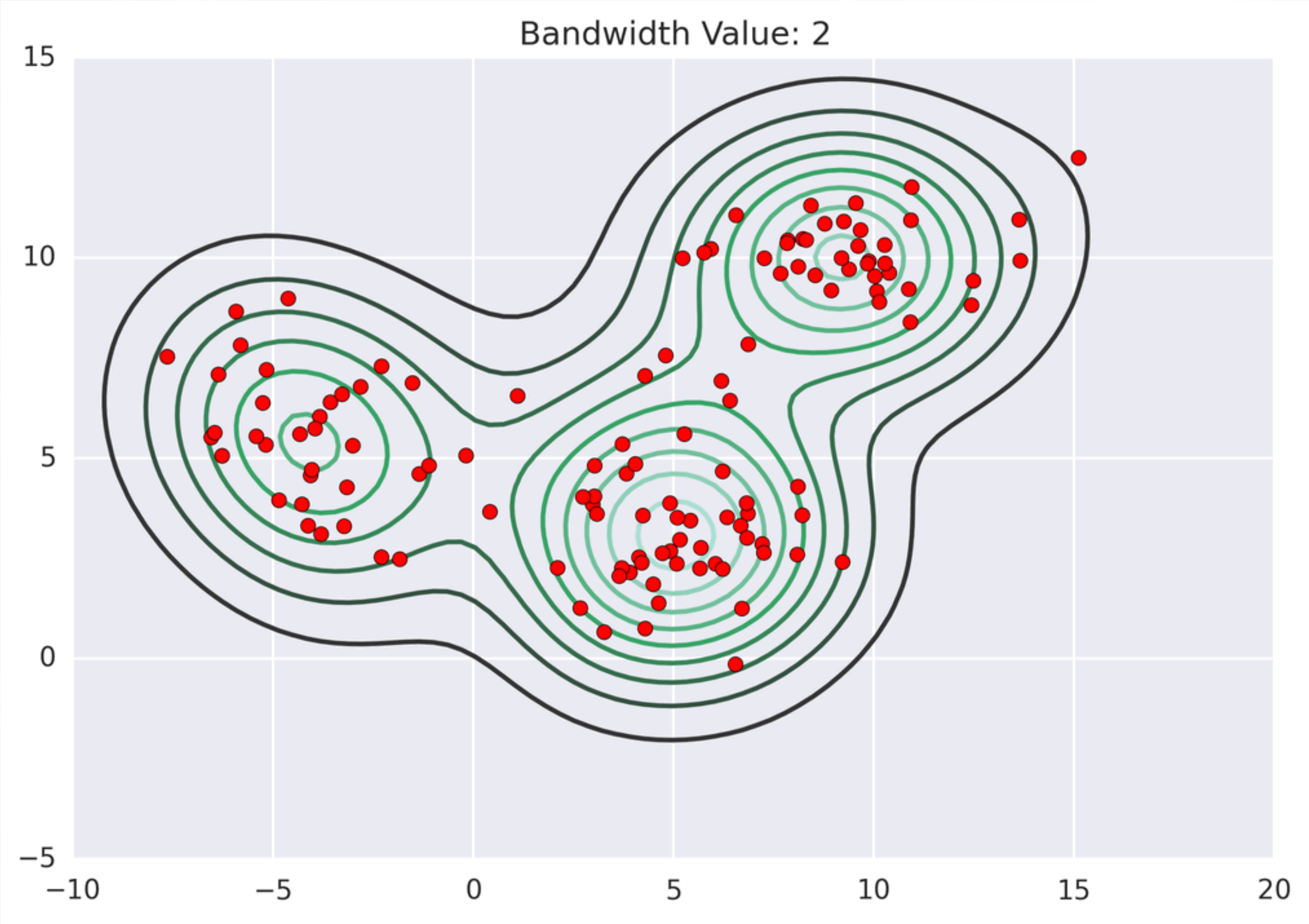
Mean-Shift Clustering: Example



Mean-Shift Clustering: Example



Mean-Shift Clustering: Example



Mean-Shift Segmentation

- An advanced and versatile technique for clustering - based segmentation

Segmented "landscape 1"

Segmented "landscape 2"

References

- D. Comaniciu, V. Ramesh, and P. Meer. Real-time tracking of non-rigid objects using mean shift. In IEEE CVPR, 2000.

- D. Comaniciu, V. Ramesh, and P. Meer. Mean shift: A robust approach towards feature space analysis. IEEE Trans. on Pattern Analysis and Machine Intelligence, 24(5):603–619, 2002.

Mean-Shift Algorithm

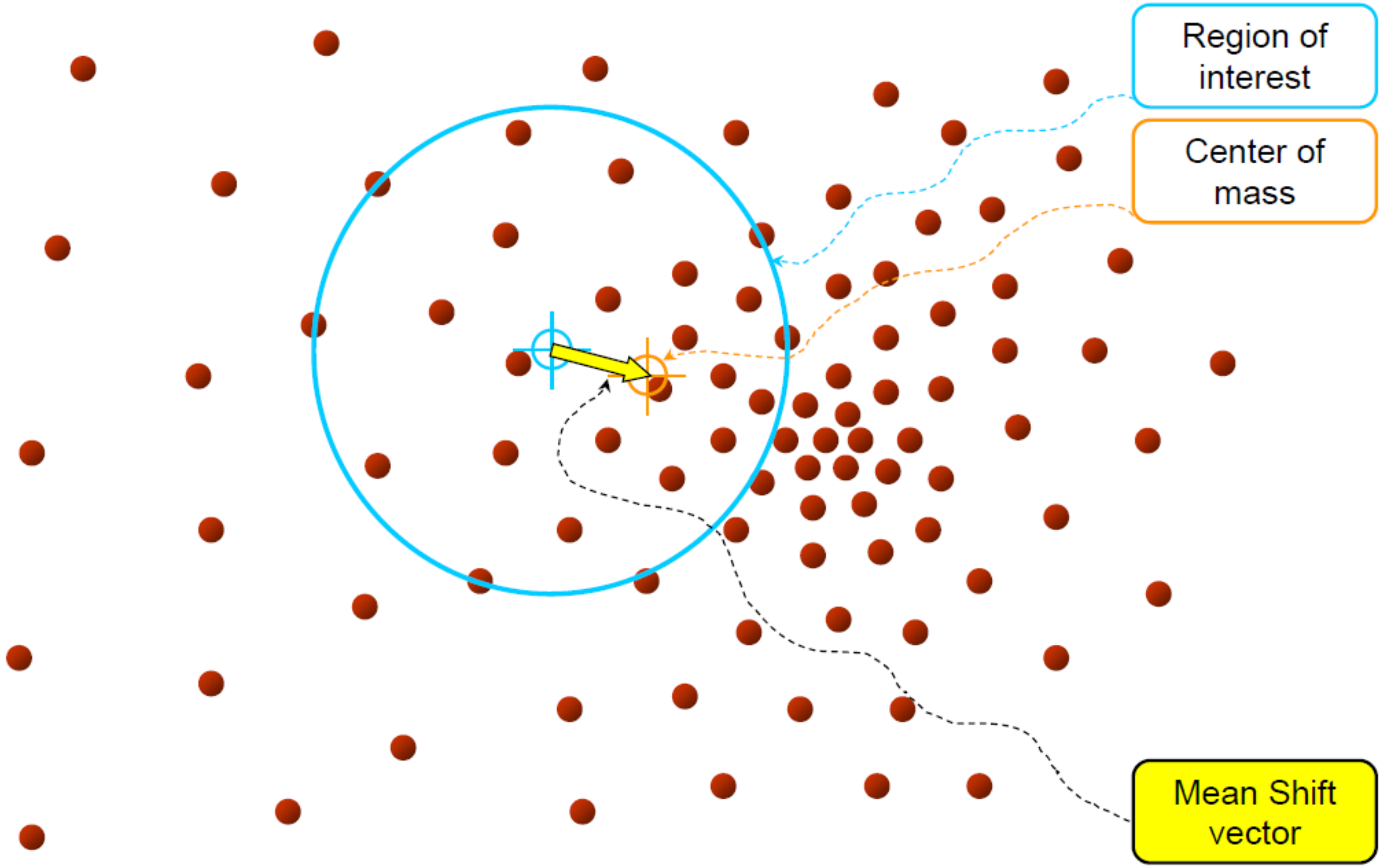
Iterative Mode Search

1. Initialize random seed, and window W

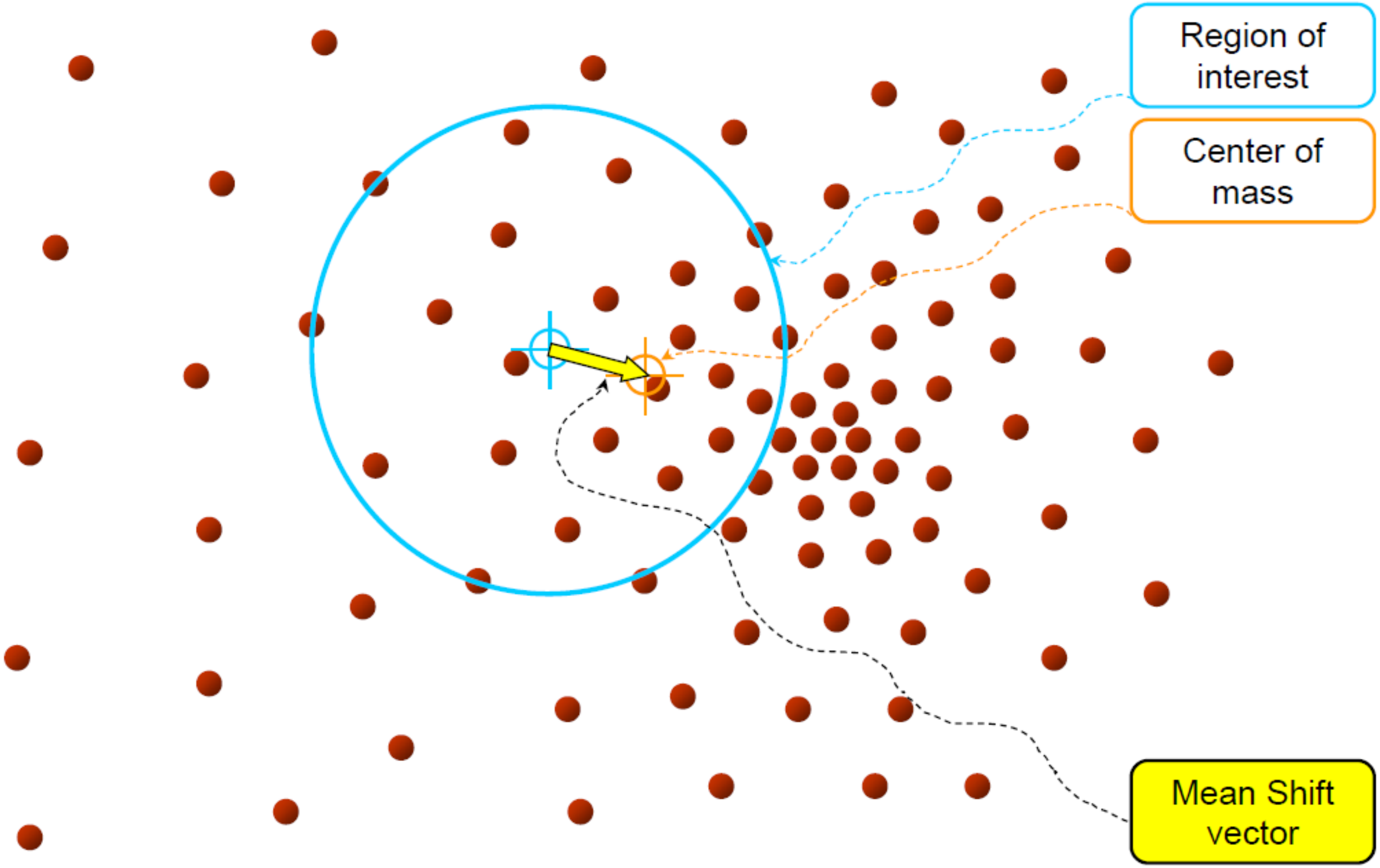
2. Calculate center of gravity (the "mean") of W: $\sum_{x \in W} xH(x)$

3. Shift the search window to the mean

4. Repeat Step 2 until convergence

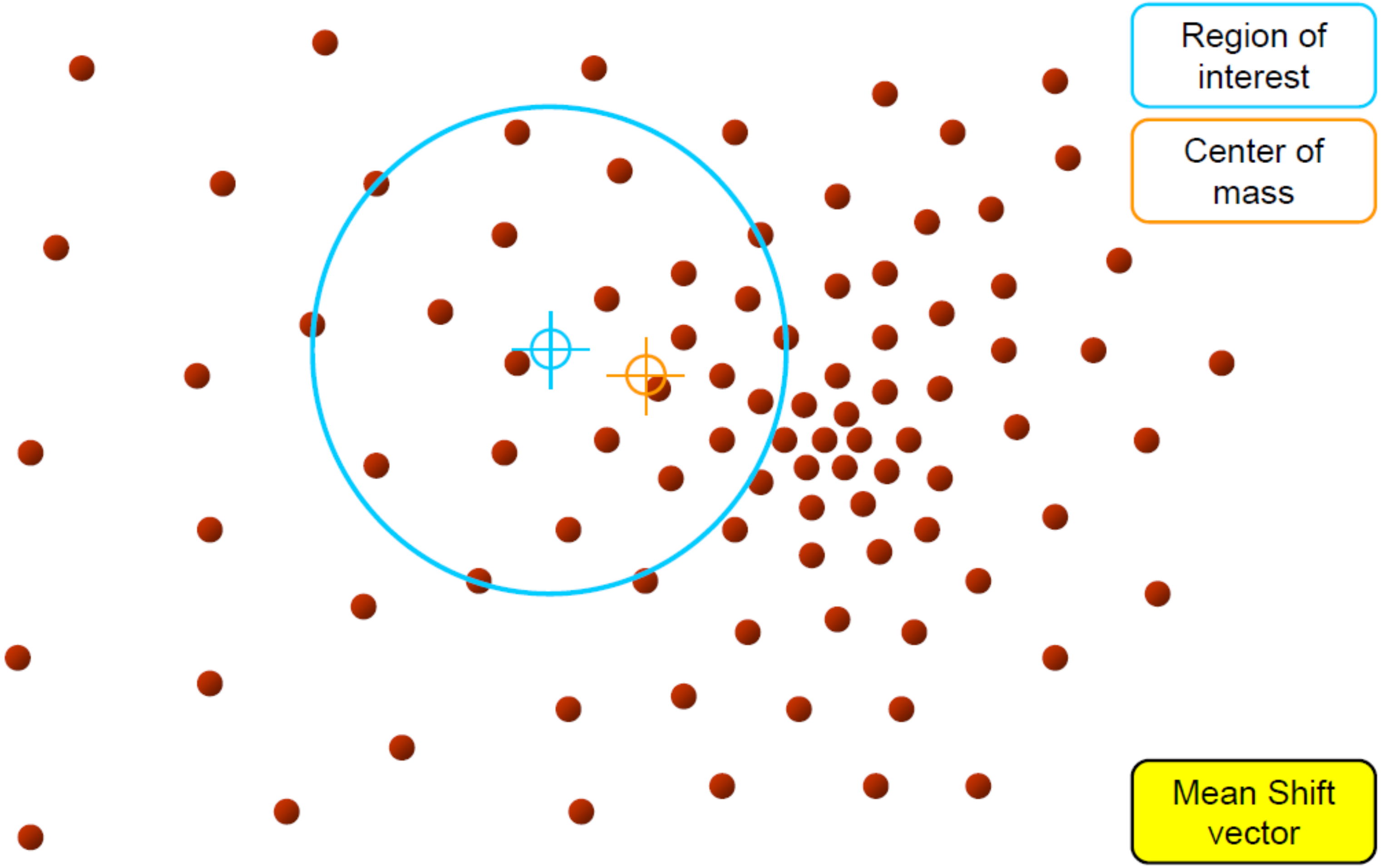


Objective : Find the densest region



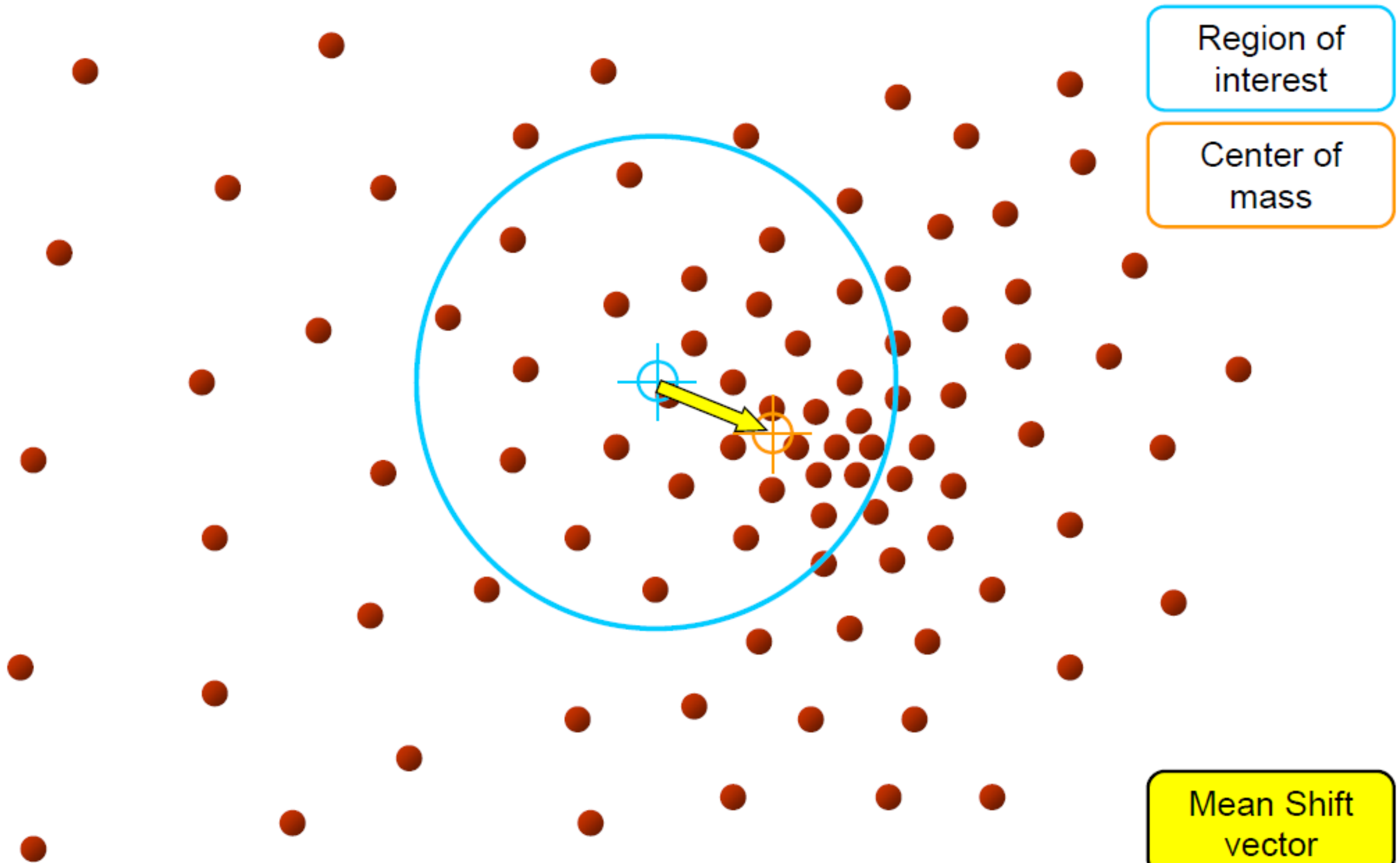
Objective : Find the densest region

Distribution of identical billiard balls



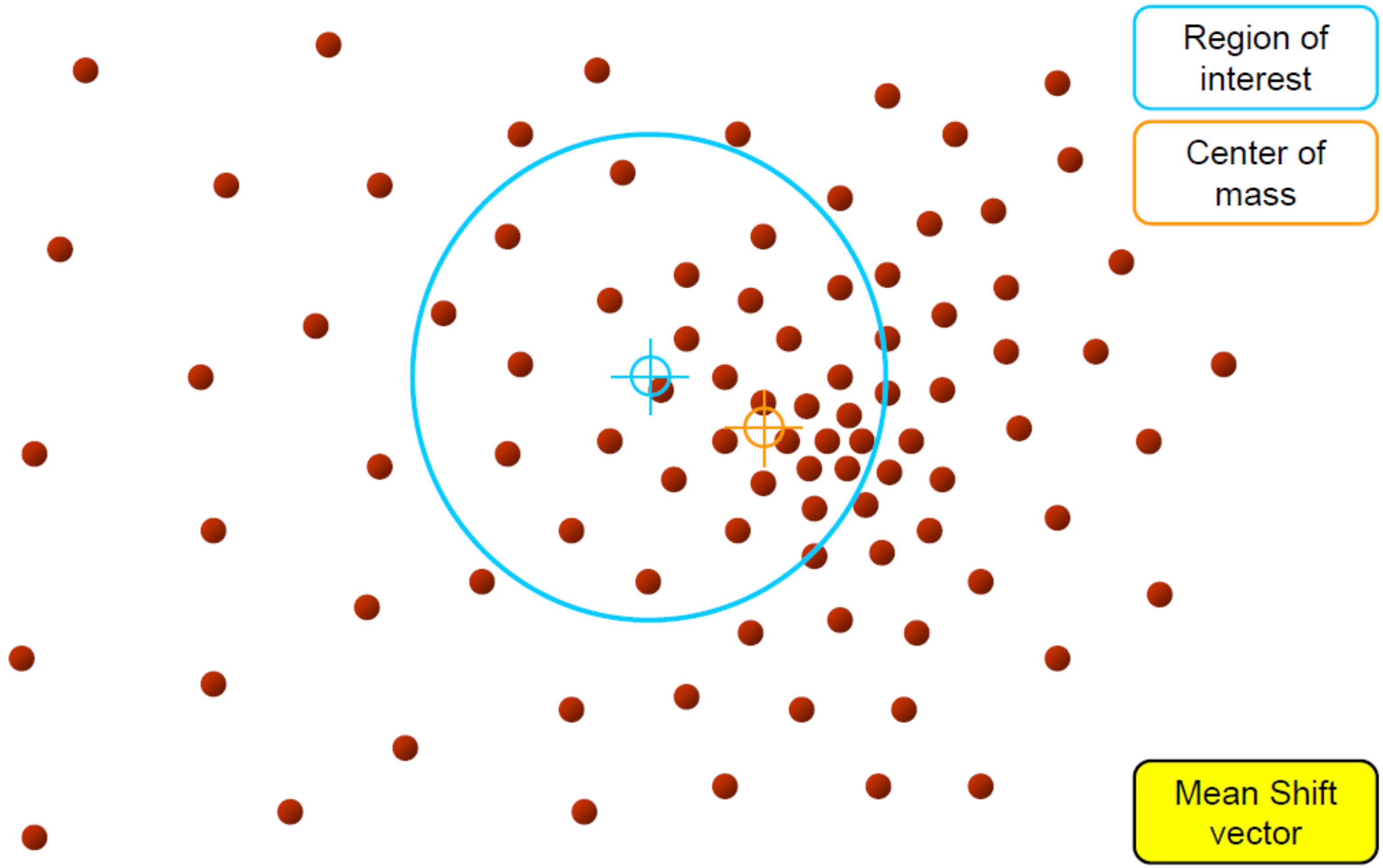
Objective : Find the densest region

Distribution of identical billiard balls



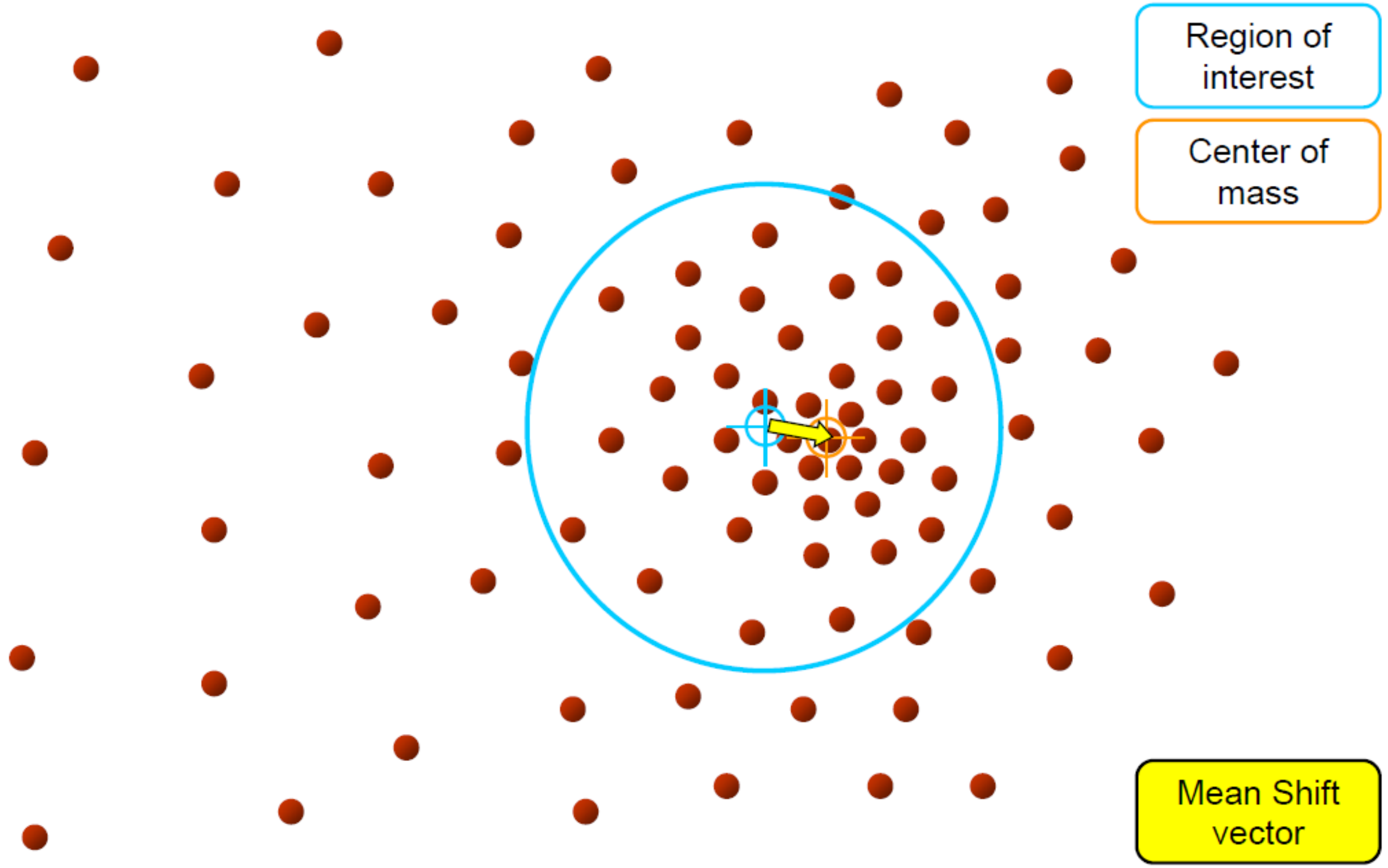
Objective : Find the densest region

Distribution of identical billiard balls

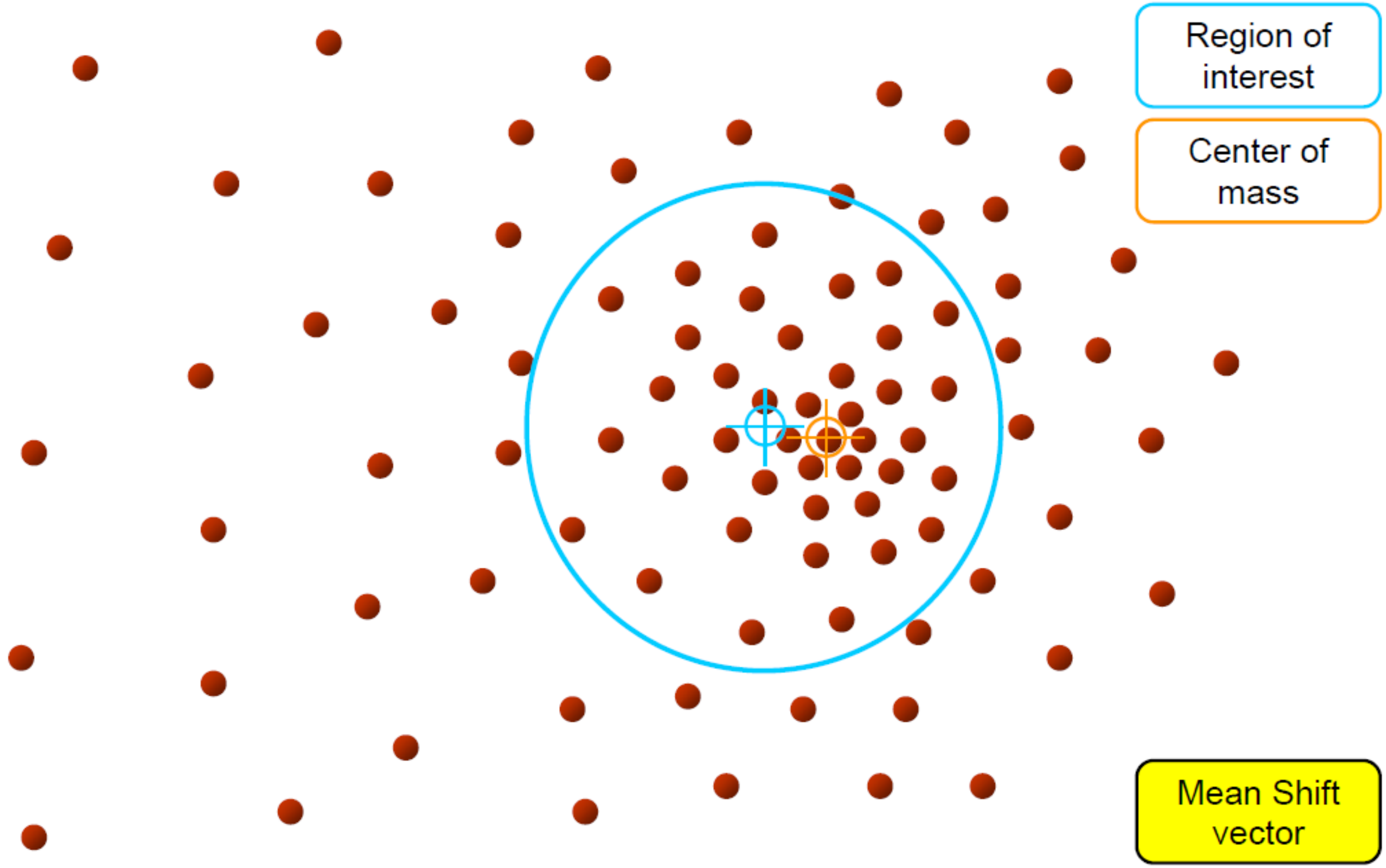


Objective : Find the densest region

Distribution of identical billiard balls

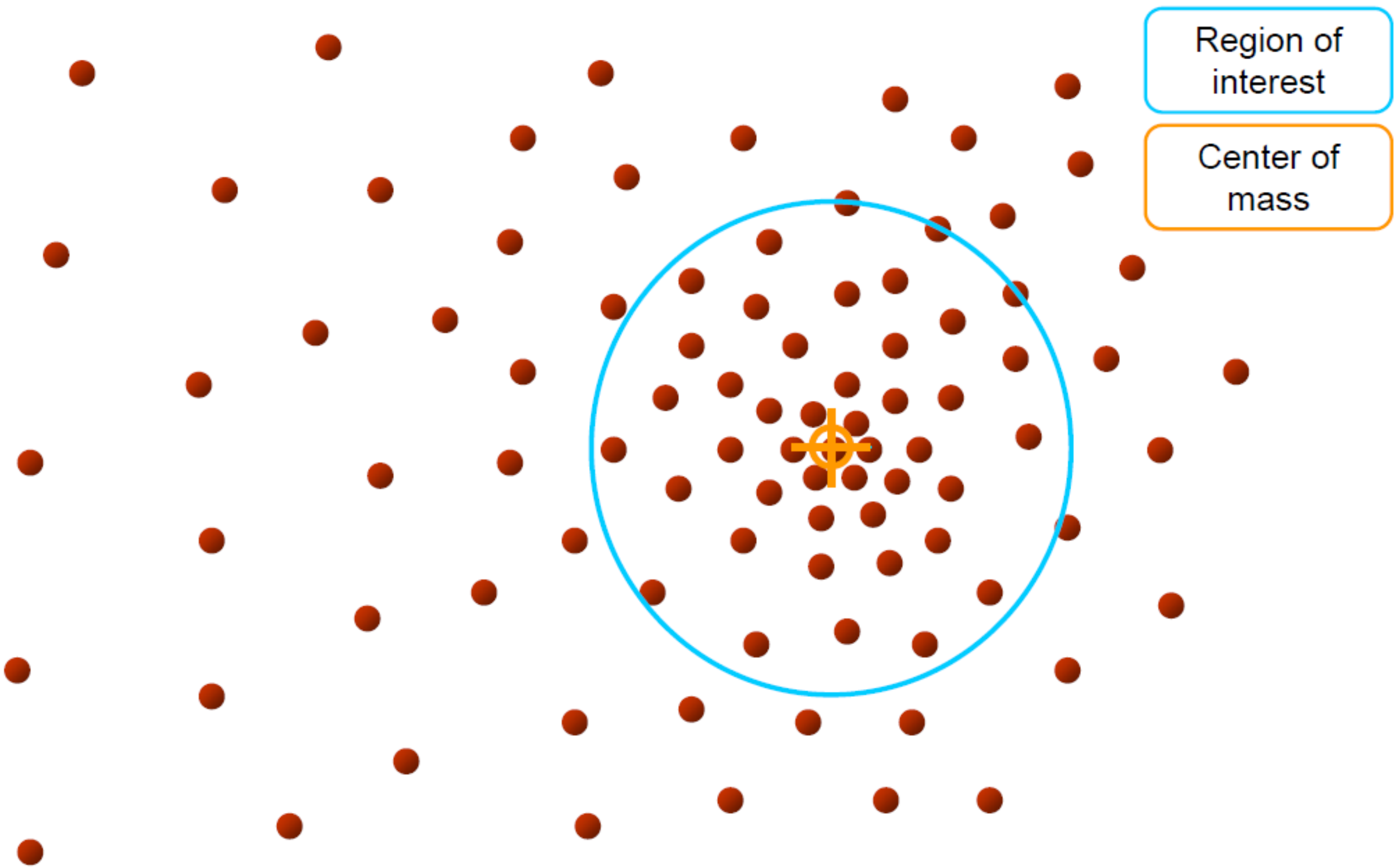


Objective : Find the densest region
Distribution of identical billiard balls



Objective : Find the densest region

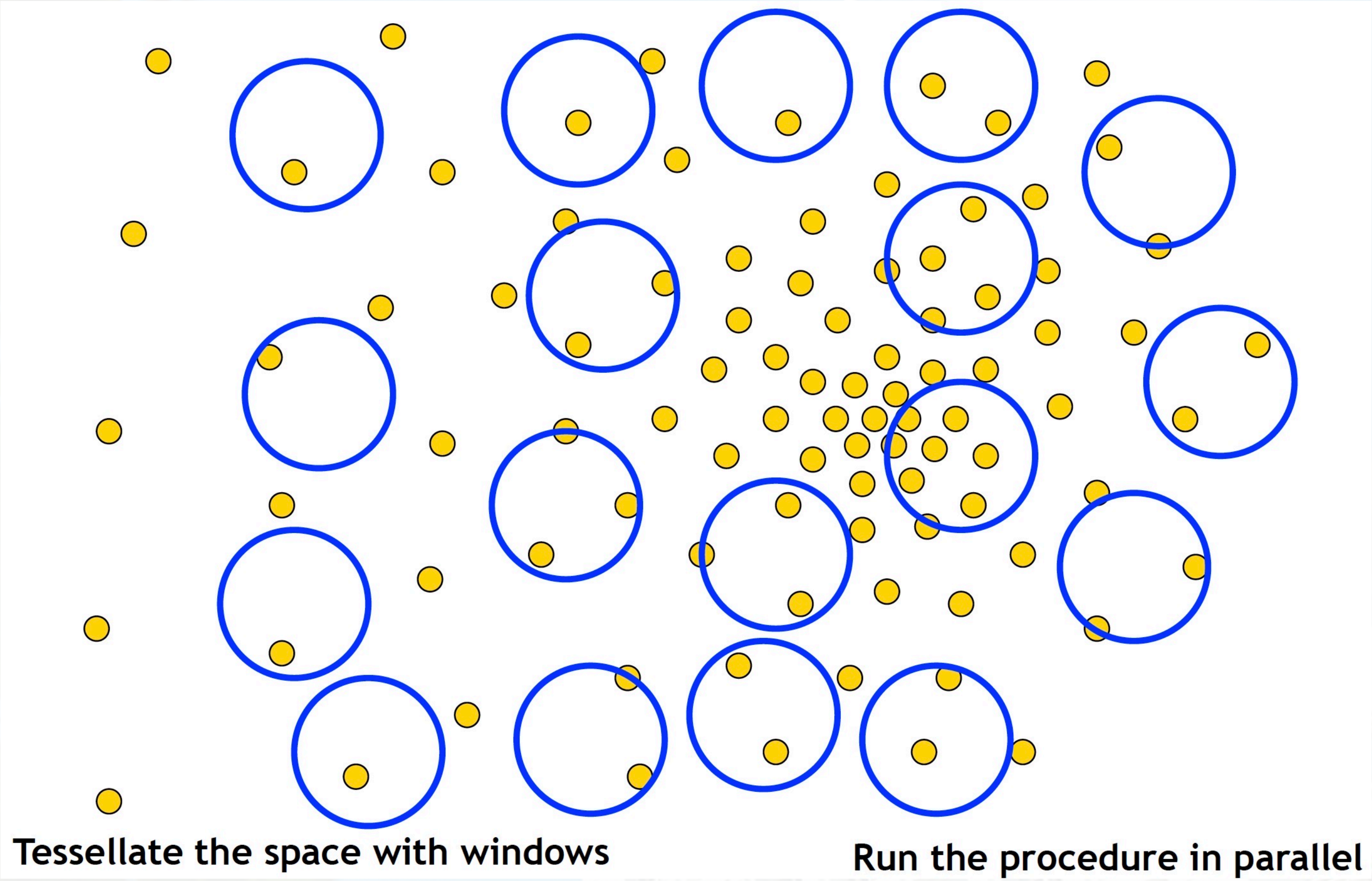
Distribution of identical billiard balls



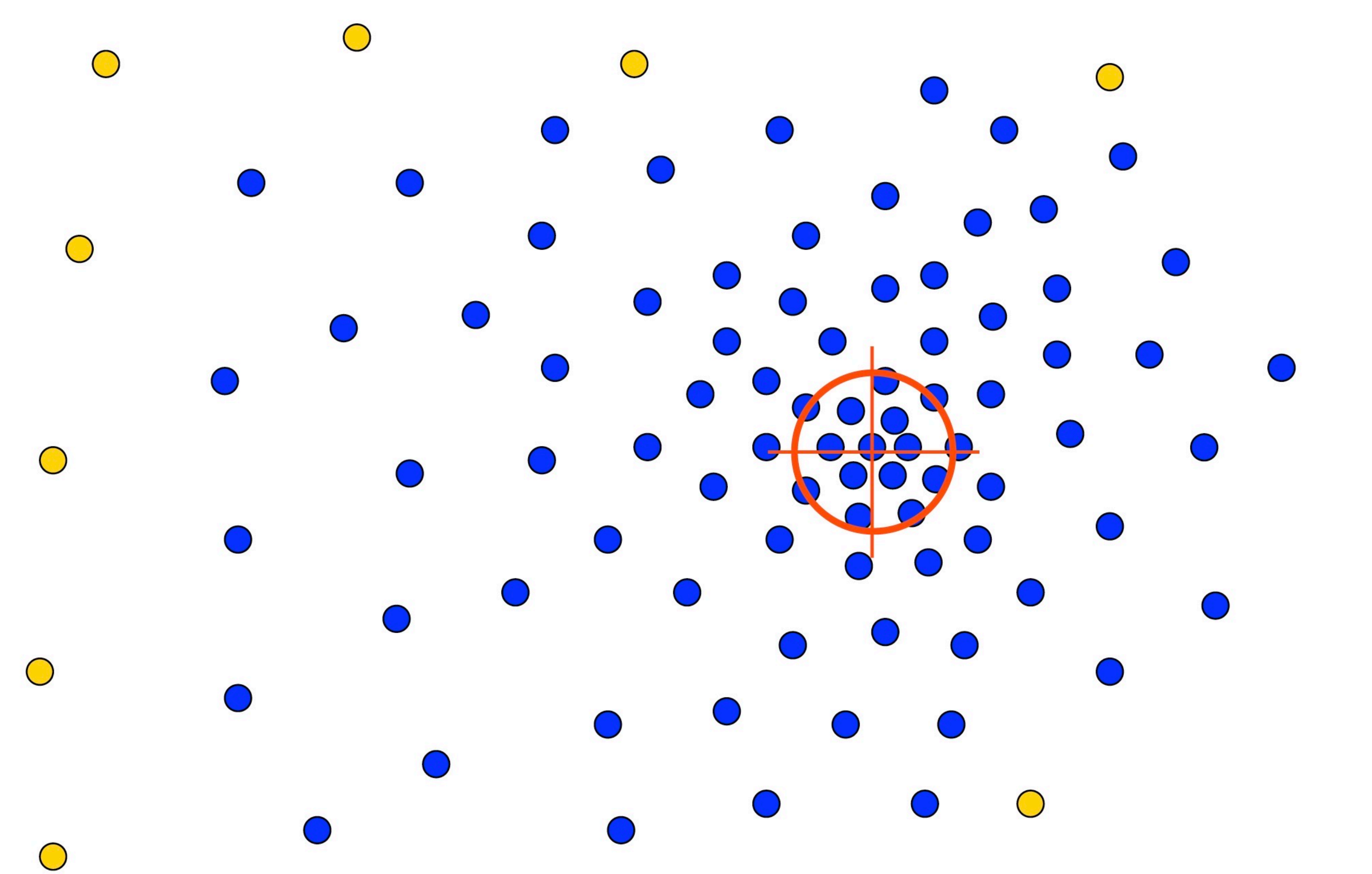
Objective : **Find the densest region**

Distribution of identical billiard balls

Real Modality Analysis



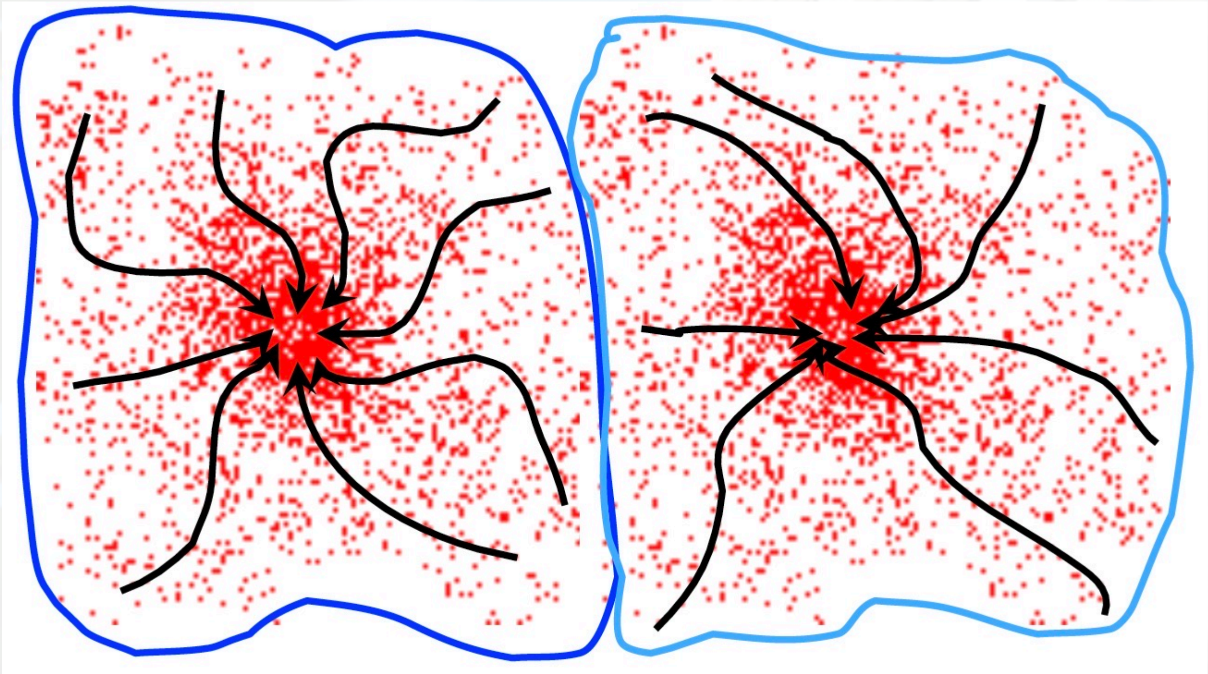
Real Modality Analysis



The blue data points were traversed by the windows towards the mode.

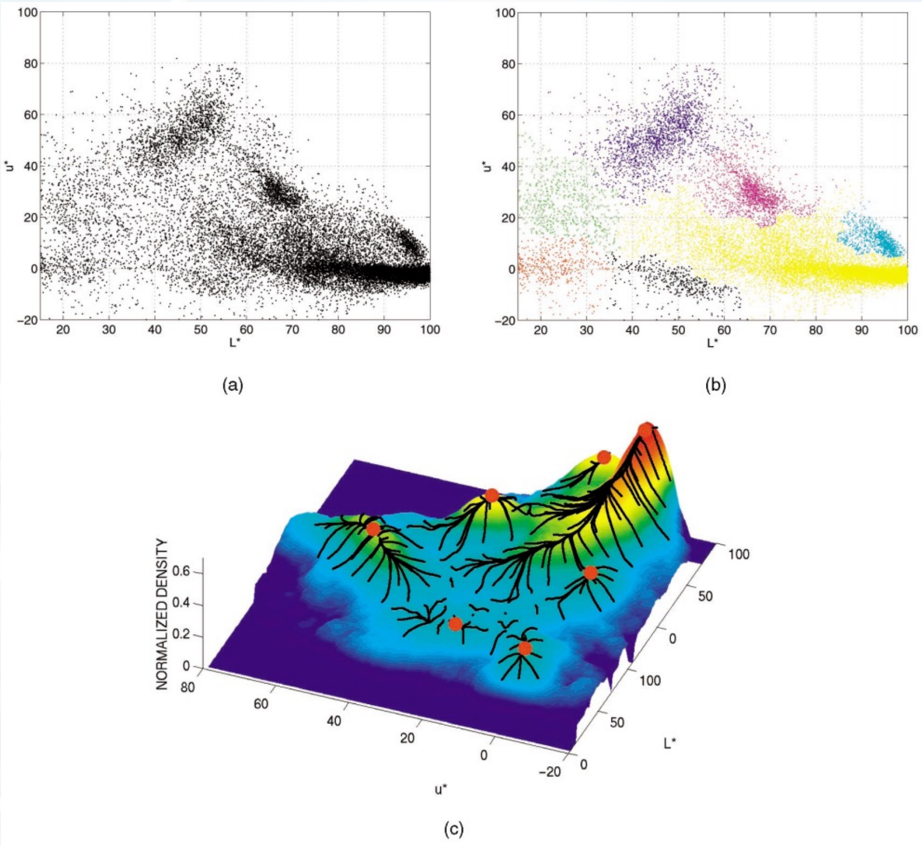
Mean-Shift Clustering

- Cluster: all data points in the attraction basin of a mode
- Attraction basin: the region for which all trajectories lead to the same mode



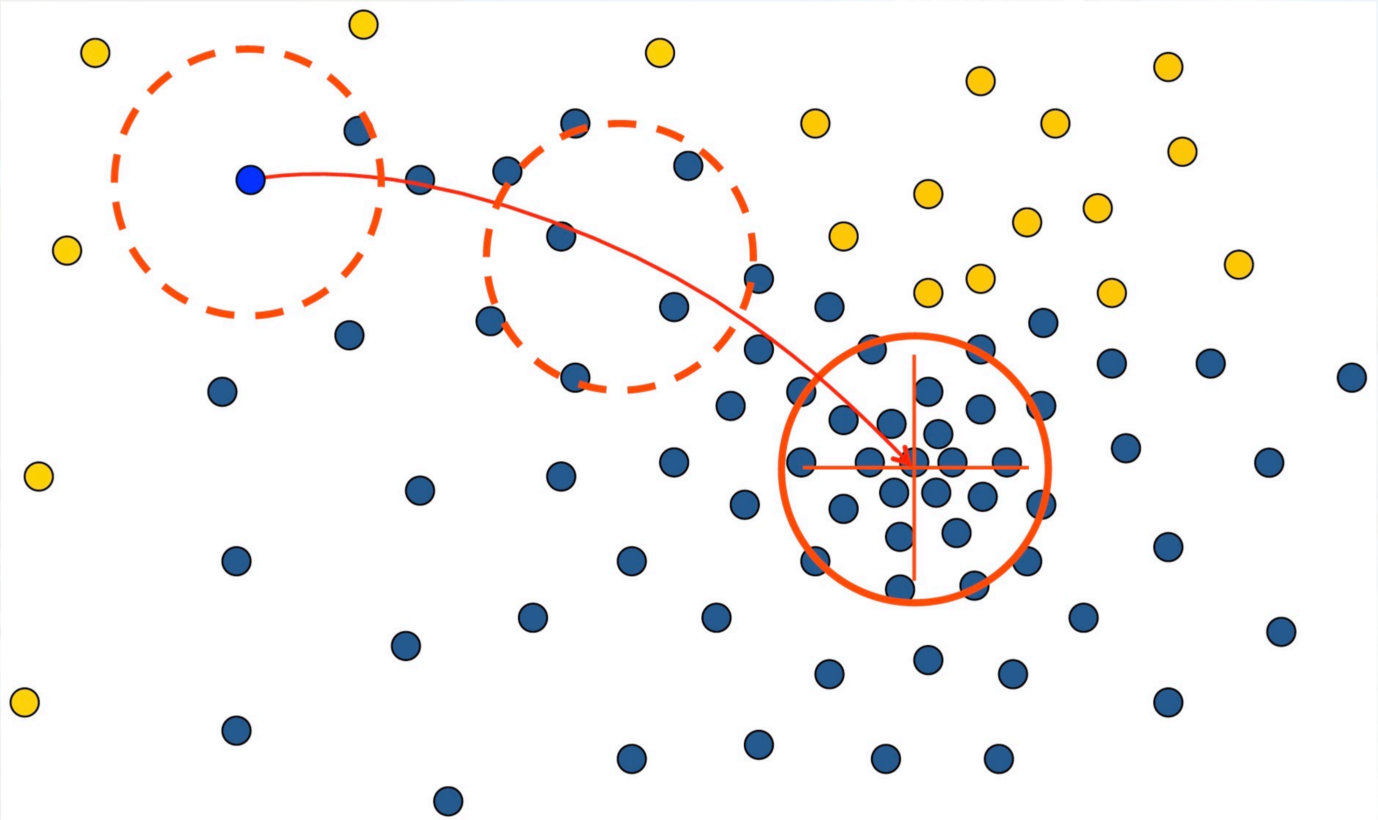
Mean-Shift Segmentation

- Find features (color, gradients, texture, etc)
- Initialize windows at individual pixel locations
- Perform mean shift for each window until convergence
- Merge windows that end up near the same “peak” or mode



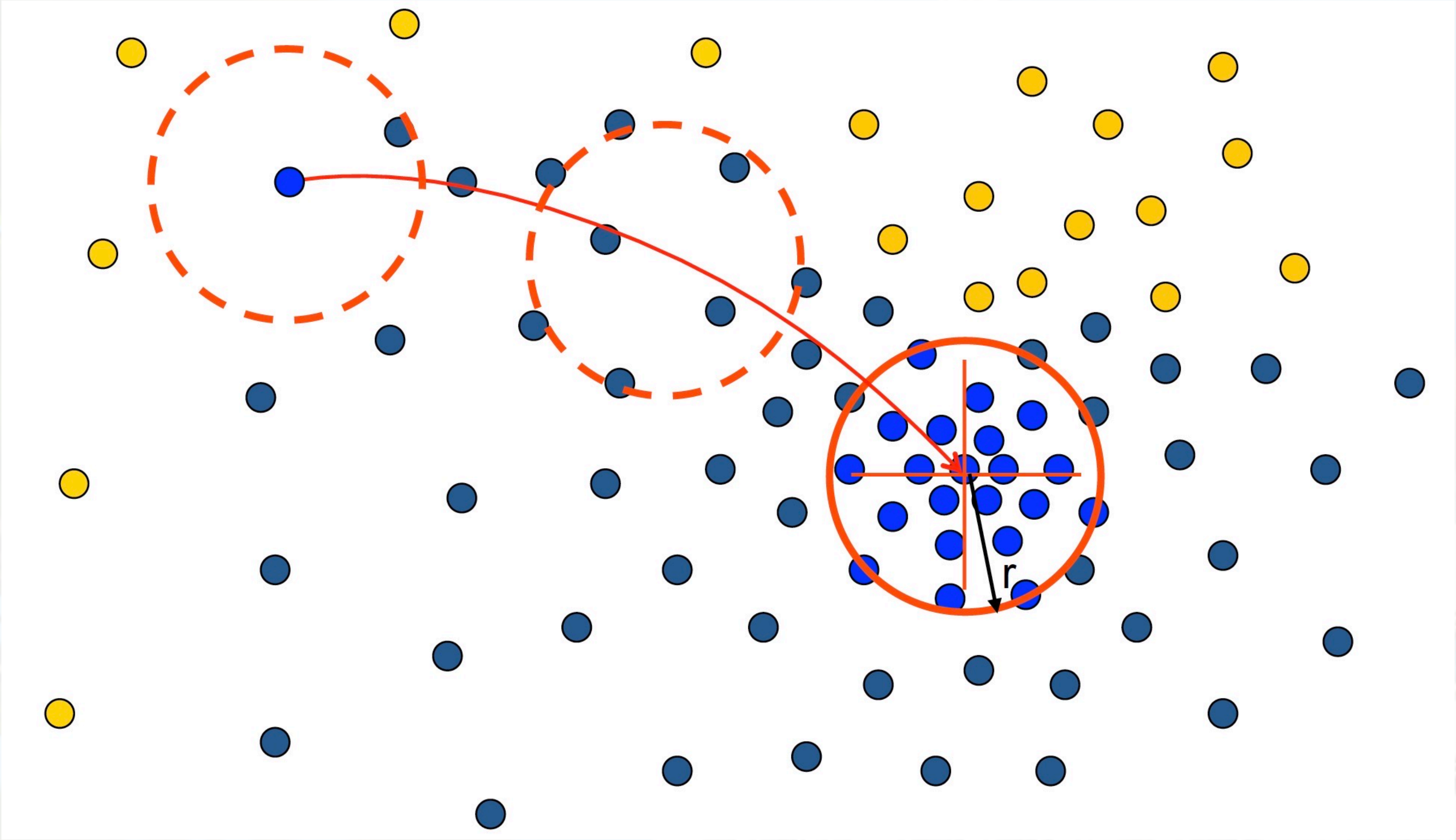
Problem: Computational Complexity

- Need to shift many windows…
- Many computations will be redundant.



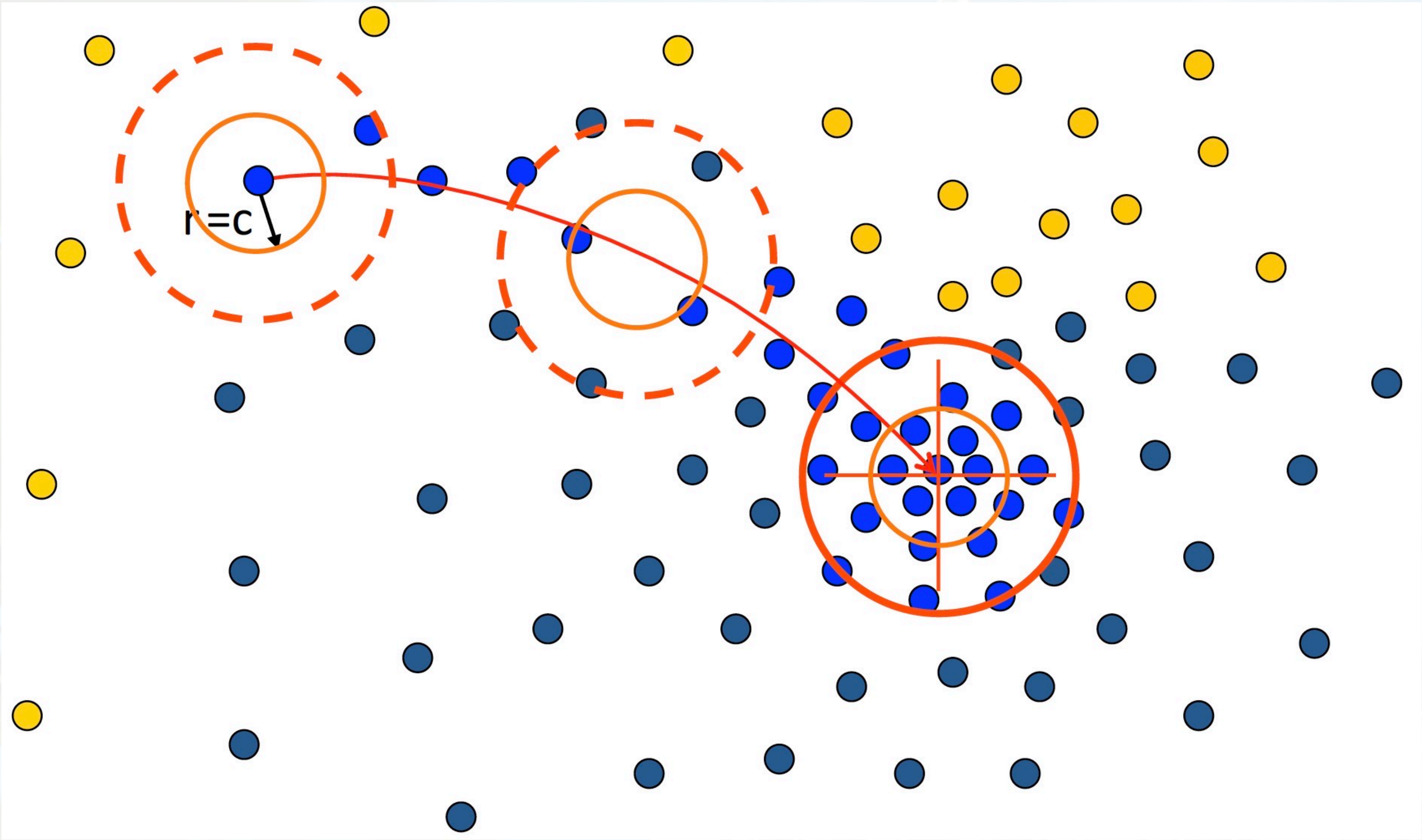
Speedups: Basin of Attractions

- Assign all points within radius r of end point to the mode



Speedups: Basin of Attractions

- Assign all points within radius r/c of the search path to the mode

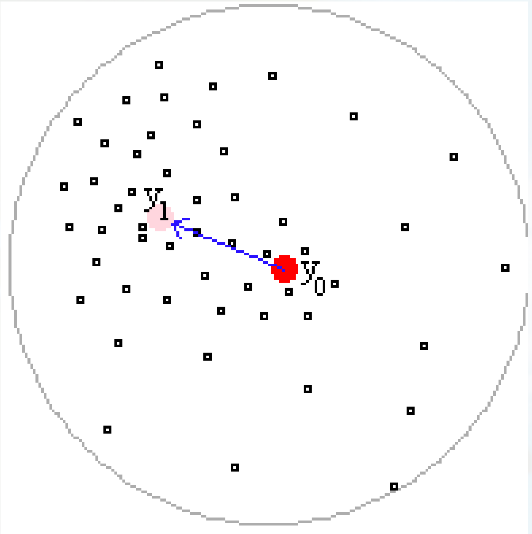


Mean Shift Vector

Given:
Data points and approximate location of the mean of this data:

Task:
Estimate the exact location of the mean of the data by determining the shift vector from the initial mean.

Mean Shift Vector Example



$$M_h(y) = \left[\frac{1}{n_x}\sum_{i=1}^{n_x} x_i\right] - y_0$$

Mean shift vector always points towards the direction of the maximum increase in the density.

Mean Shift (Weighted)

$$M_h(y_0) = \left[\frac{\sum_{i=1}^{n_x} w_i(y_0) x_i}{\sum_{i=1}^{n_x} w_i(y_0)} \right] - y_0$$

n_x : number of points in the kernel
$\mathbf{y}_0$: initial mean location
$\mathbf{x}_i$: data points
h : kernel radius

Weights are determined using kernels (masks):
Uniform, Gaussian or Epanechnikov

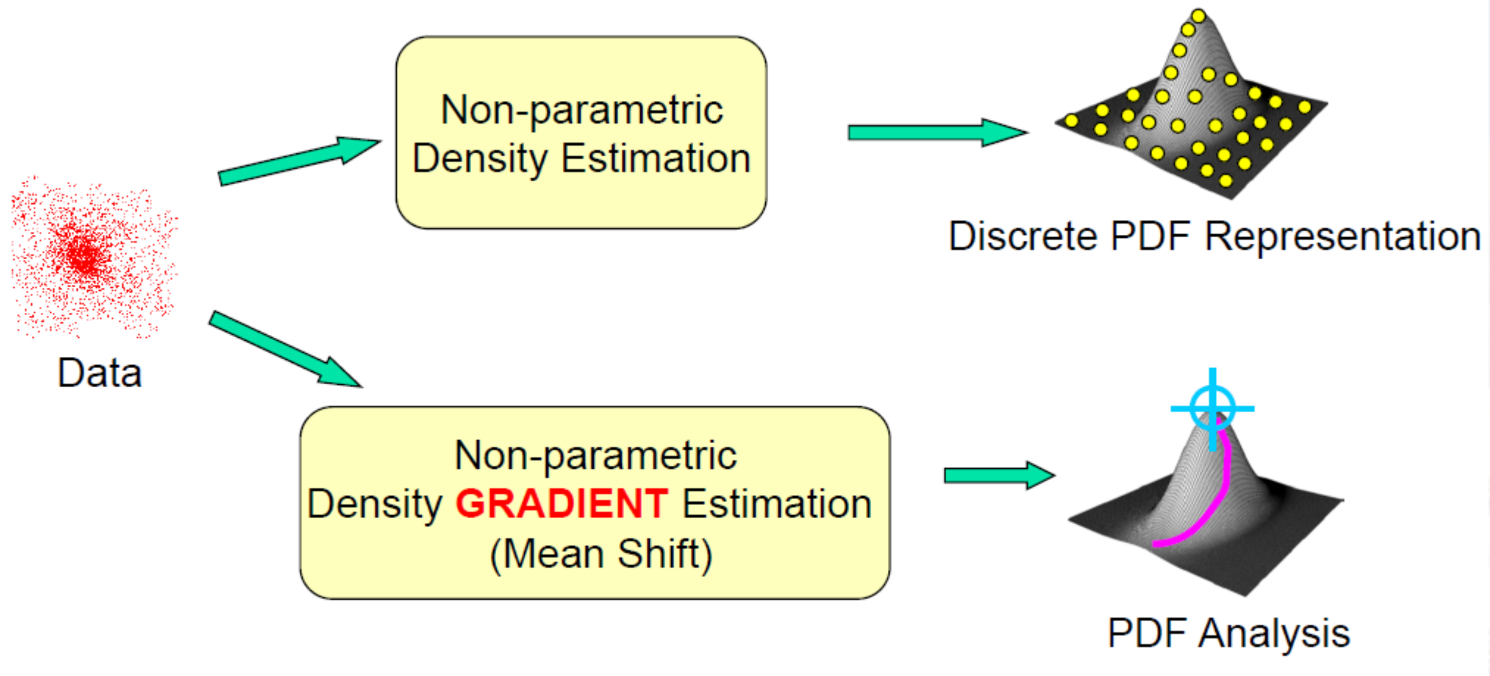
Properties of Mean Shift

- Mean shift vector has the direction of the gradient of the density estimate.

- It is computed iteratively for obtaining the maximum density in the local neighborhood.

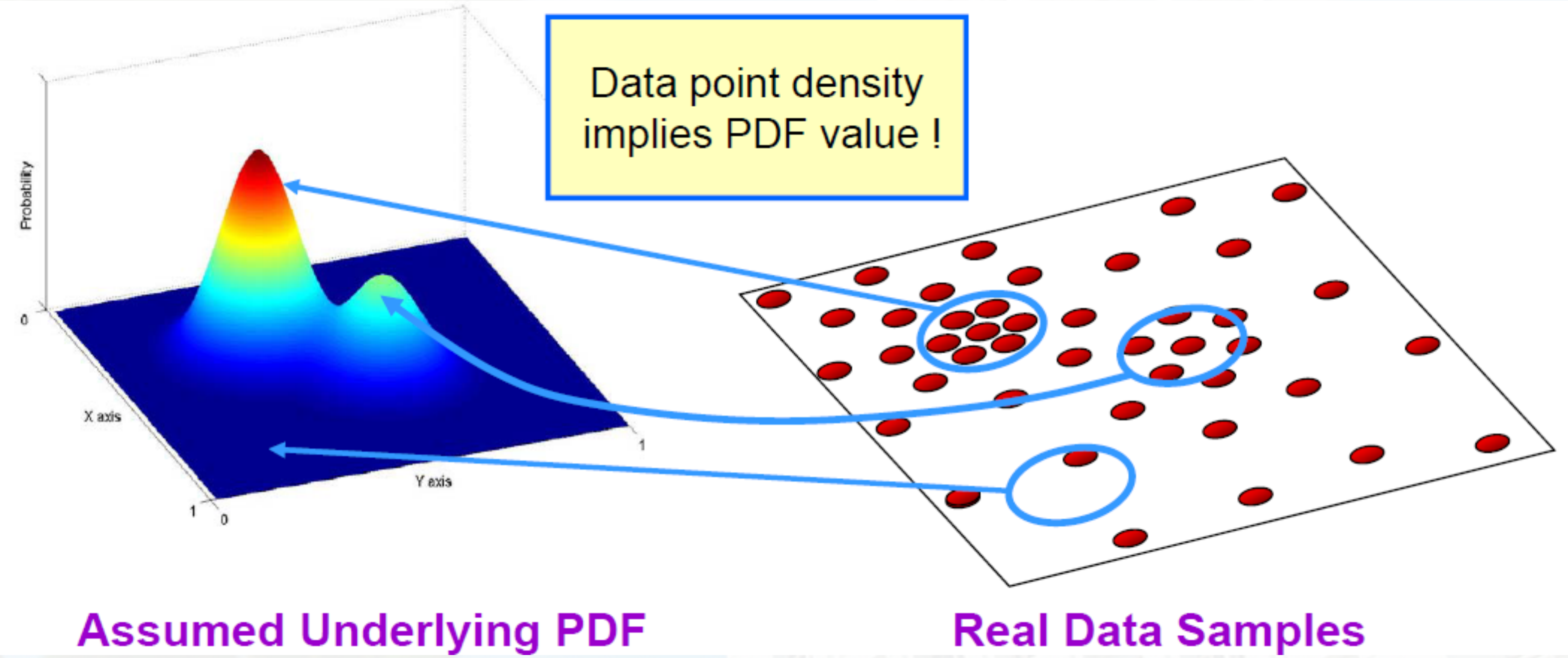
What is Mean-Shift?

- A tool for finding modes in a set of data samples, manifesting an underlying probability density function (PDF) in R^N

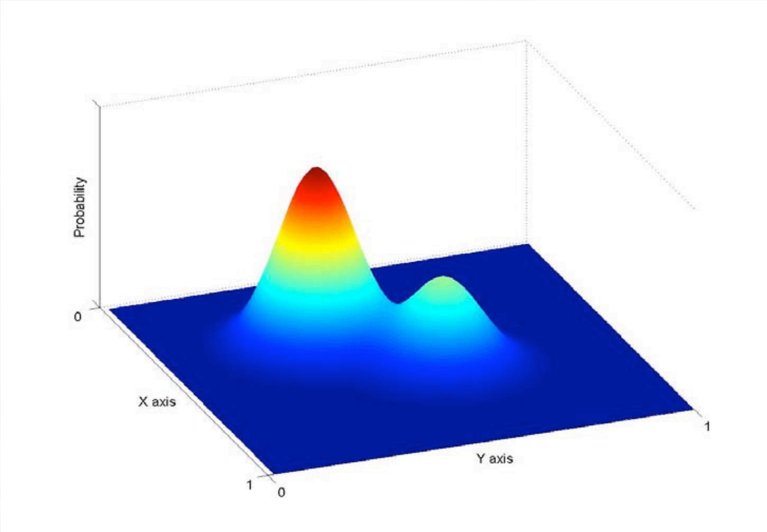


Non-Parametric Density Estimation

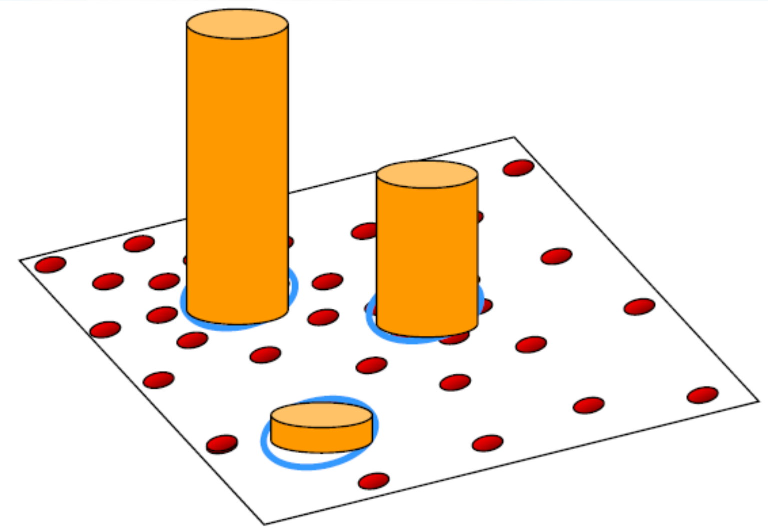
Assumption : The data points are sampled from an underlying PDF



Non-Parametric Density Estimation

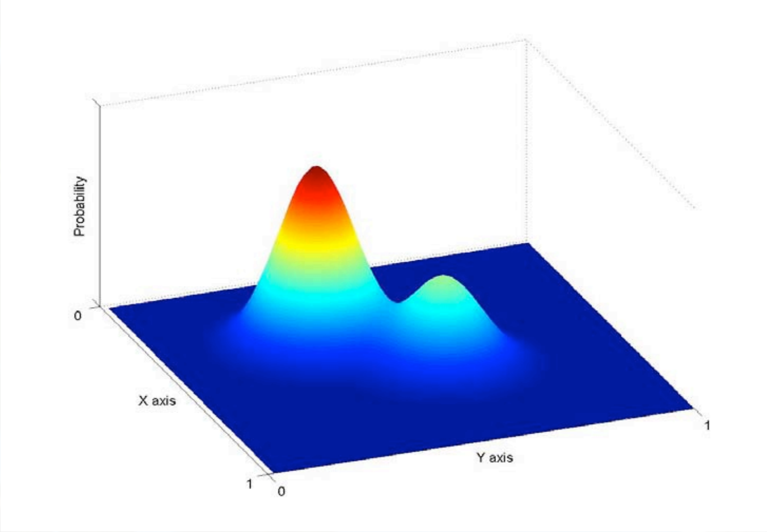


Assumed Underlying PDF

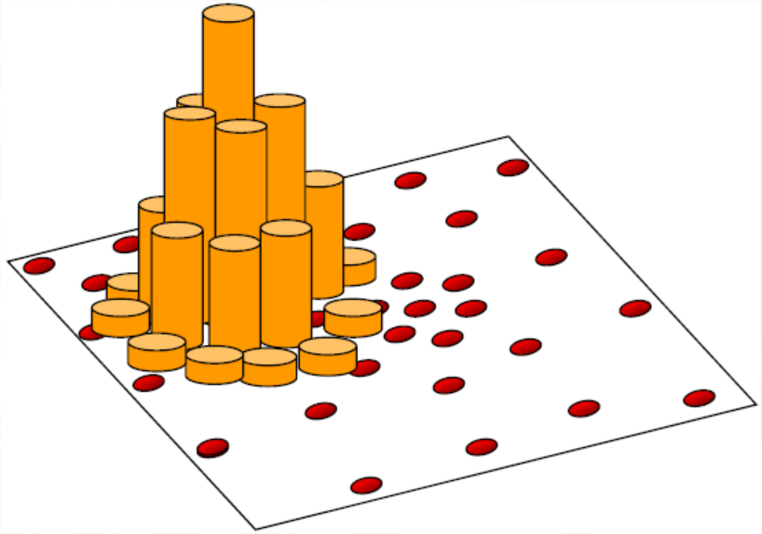


Real Data Samples

Non-Parametric Density Estimation



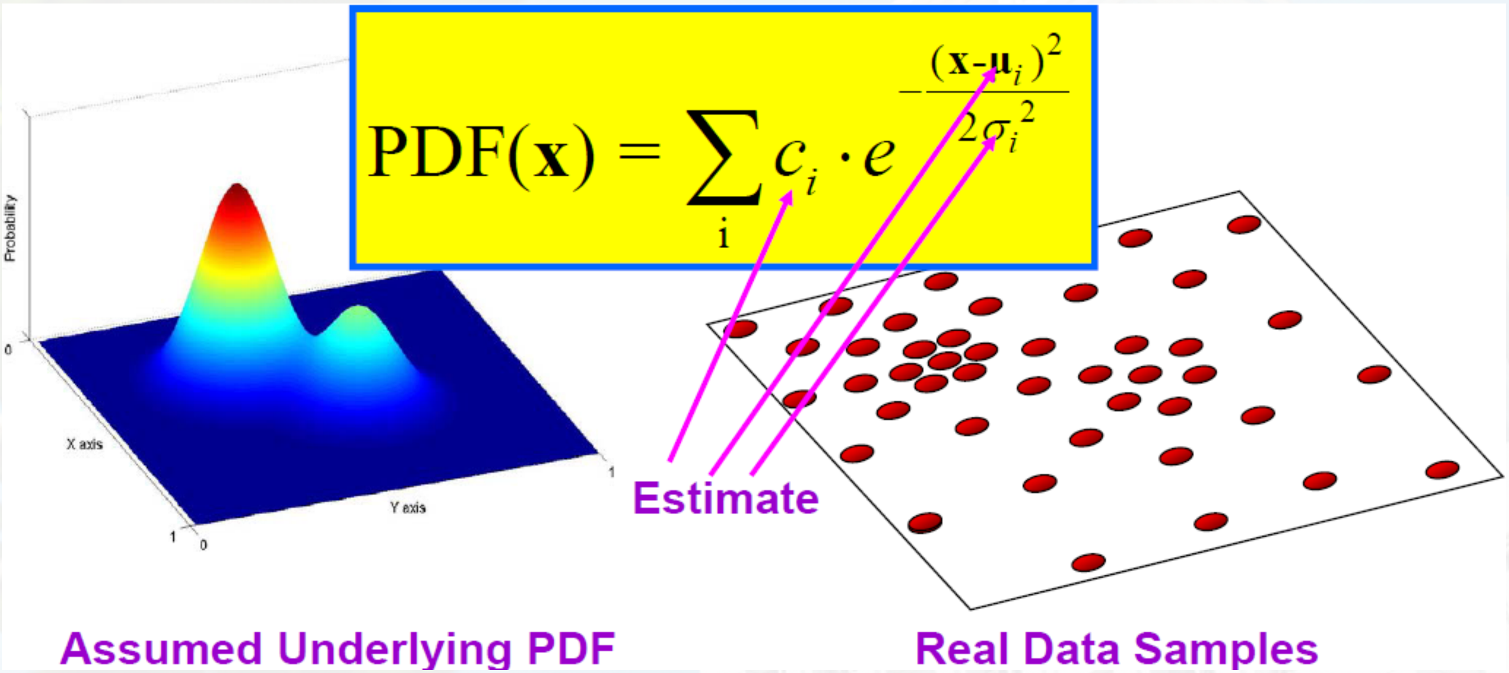
Assumed Underlying PDF



Real Data Samples

Parametric Density Estimation

Assumption : The data points are sampled from an underlying PDF

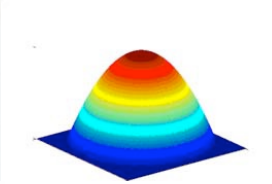


Kernel Density Estimation

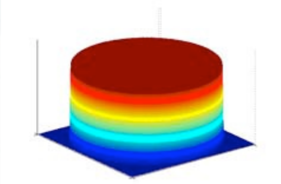
$$P(x) = \frac{1}{n}\sum_{i=1}^{n} K(x - x_i)$$

Examples:

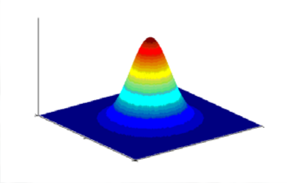
- Epanechnikov Kernel $K_E(x) = \begin{cases} c(1 - \|x\|^2) & \|x\| < 1 \\ 0 & otherwise \end{cases}$



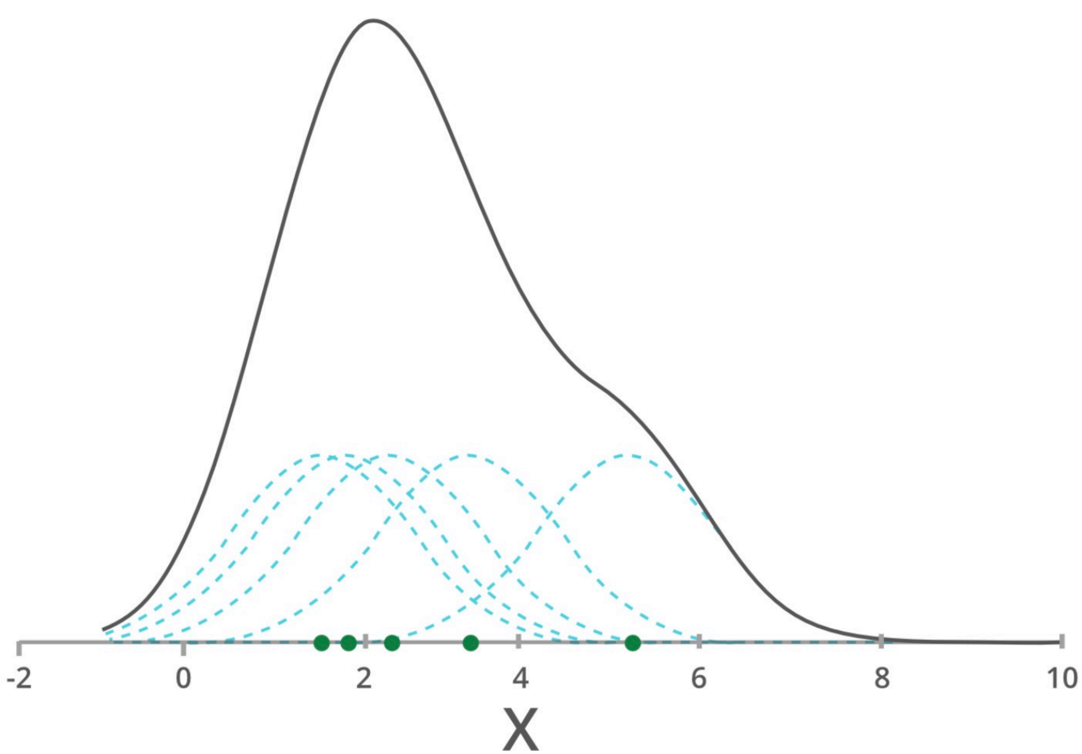
- Uniform Kernel $K_U(x) = \begin{cases} c & \|x\| < 1 \\ 0 & otherwise \end{cases}$



- Normal Kernel $K_N(x) = c \cdot exp\left(-\frac{1}{2}\|x\|^2\right)$



Kernel Density Estimation



Example of kernel density estimation using a gussain kernel for each data point: Adding up small Gauggians about each example returns our net estimate for the total density, the black curve.

Kernel Density Estimation

Radially symmetric Kernel

$$K(x) = ck(\|x\|^2)$$

Kernel density estimate

$$P(\mathrm{x}) = \frac{1}{n}\sum_{i=1}^{n} K(\mathrm{x}\text{-}\mathrm{x}_i) = \frac{1}{n}c\sum_{i=1}^{n} k\left(\|\mathrm{x}\text{-}\mathrm{x}_i\|^2\right)$$

Kernel Density Estimation

$$P(x) = \frac{1}{n} c \sum_{i=1}^{n} k(\|x - x_i\|^2)$$

$$\nabla P(x) = \frac{1}{n} c \sum_{i=1}^{n} \nabla k(\|x - x_i\|^2)$$

$$\nabla P(x) = \frac{1}{n} 2c \sum_{i=1}^{n} (x - x_i) k'(\|x - x_i\|^2)$$

Kernel Density Estimation

$$\nabla P(x) = \frac{1}{n} 2c \sum_{i=1}^{n} (x - x_i) k'(\|x - x_i\|^2)$$

$$\nabla P(x) = \frac{1}{n} 2c \sum_{i=1}^{n} (x_i - x) g(\|x - x_i\|^2)$$

$$\nabla P(x) = \frac{1}{n} 2c \sum_{i=1}^{n} x_i g(\|x - x_i\|^2) - \frac{1}{n} 2c \sum_{i=1}^{n} x g(\|x - x_i\|^2)$$

$$\nabla P(x) = \frac{1}{n} 2c \sum_{i=1}^{n} g(\|x - x_i\|^2) \left[\frac{\sum_{i=1}^{n} x_i g(\|x - x_i\|^2)}{\sum_{i=1}^{n} g(\|x - x_i\|^2)} - x \right] \quad g(x) = k'(x)$$

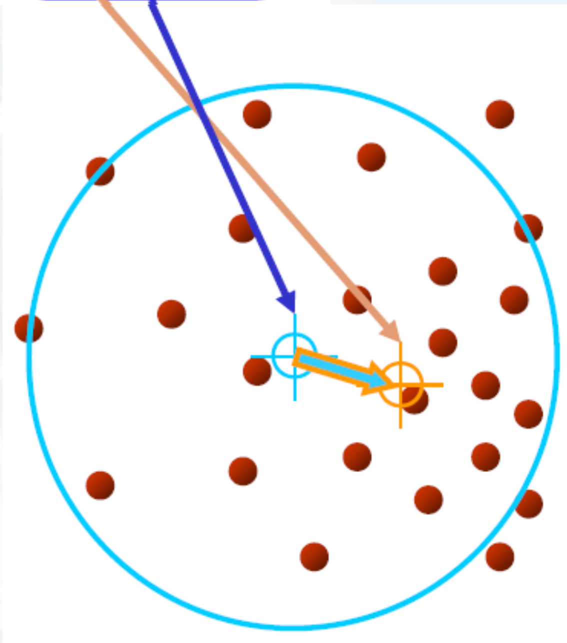
Kernel Density Estimation

$$\nabla P(x) = \frac{1}{n} 2c \sum_{i=1}^{n} g(\|x - x_i\|^2) \left[\frac{\sum_{i=1}^{n} x_i g(\|x - x_i\|^2)}{\sum_{i=1}^{n} g(\|x - x_i\|^2)} - x \right]$$

$$\nabla P(x) = \frac{1}{n} 2c \sum_{i=1}^{n} g_i \left[\frac{\sum_{i=1}^{n} x_i g_i}{\sum_{i=1}^{n} g_i} - x \right]$$

Computing the Mean Shift

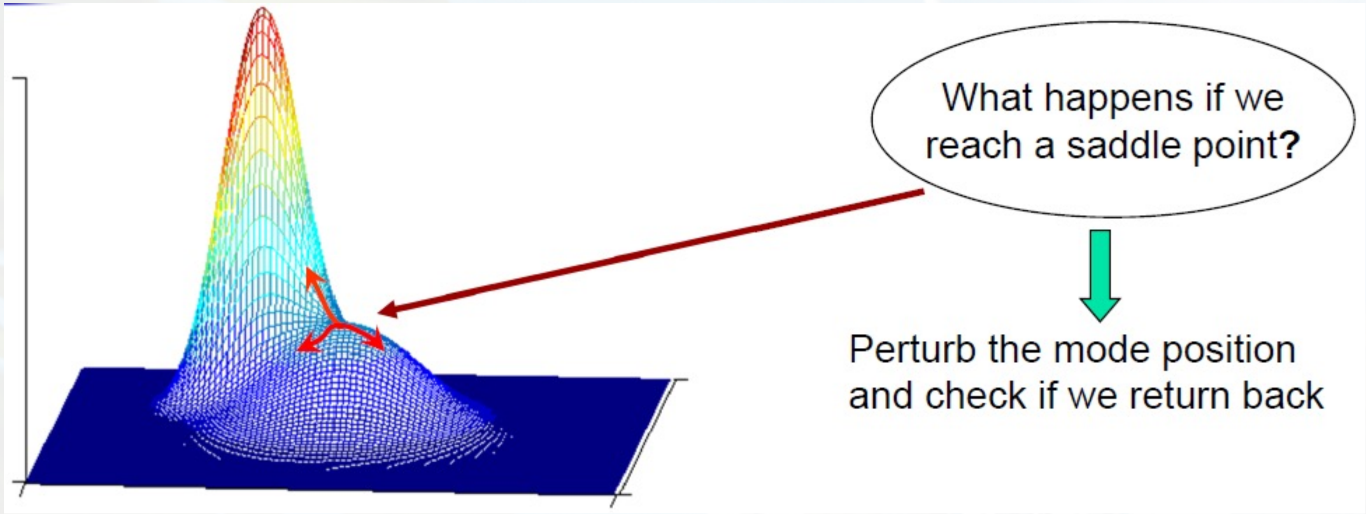
$$\nabla P(\mathbf{x}) = \frac{c}{n}\sum_{i=1}^{n} \nabla k_i = \frac{c}{n}\left[\sum_{i=1}^{n} g_i\right]\left[\frac{\sum_{i=1}^{n} \mathbf{x}_i g_i}{\sum_{i=1}^{n} g_i} - \mathbf{x}\right]$$



$$\nabla P(x) = \frac{c}{n}\sum_{i=1}^{n} g_i \times m(x)$$

$$m(x) = \frac{\nabla P(x)}{\frac{c}{n}\sum_{i=1}^{n} g_i}$$

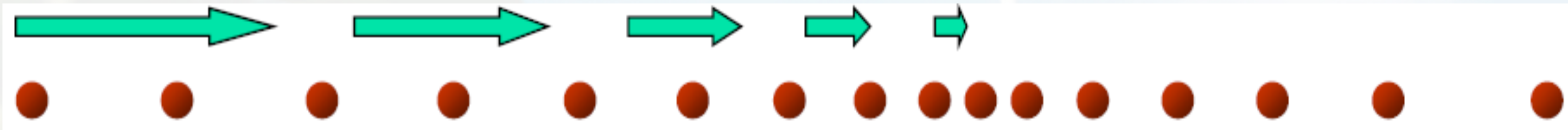
Mean Shift Mode Detection



Updated Mean Shift Procedure:

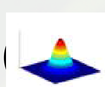
- Find all modes using the Simple Mean Shift Procedure
- Prune modes by perturbing them (find saddle points and plateaus)
- Prune nearby – take highest mode in the window

Mean Shift Properties



- Automatic convergence speed
  - mean shift vector size depends on gradient.
- Near maxima, the steps are small and refined

Adaptive Gradient Ascent

- Convergence is guaranteed for infinitesimal steps only, (therefore set a lower bound)
- For Uniform Kernel (), convergence is achieved in a finite number of steps
- Normal Kernel () exhibits a smooth trajectory, but is slower than Uniform Kernel ()

Mean Shift Pros and Cons

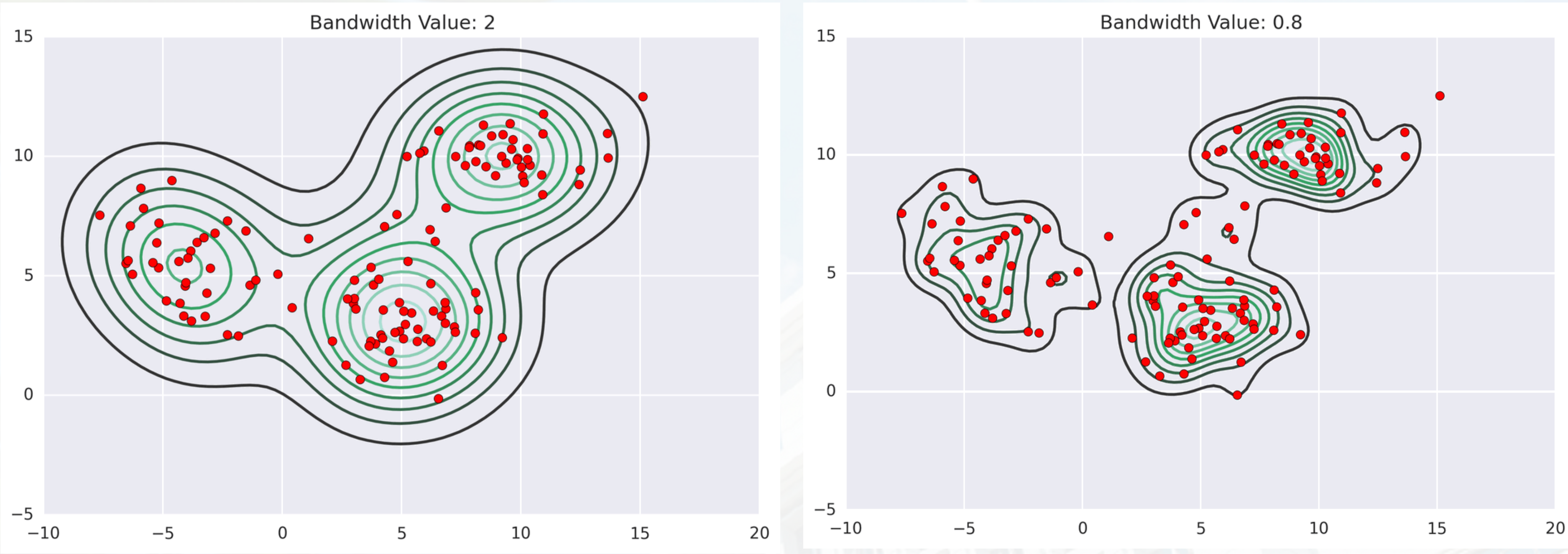
Pros:

- Finds variable number of modes
- Robust to outliers
- General, application-independent tool
- Model-free, doesn’t assume any prior shape like spherical, elliptical, etc. on data clusters
- Just a single parameter (window size h) where h has a physical meaning (unlike k-means)

Cons:

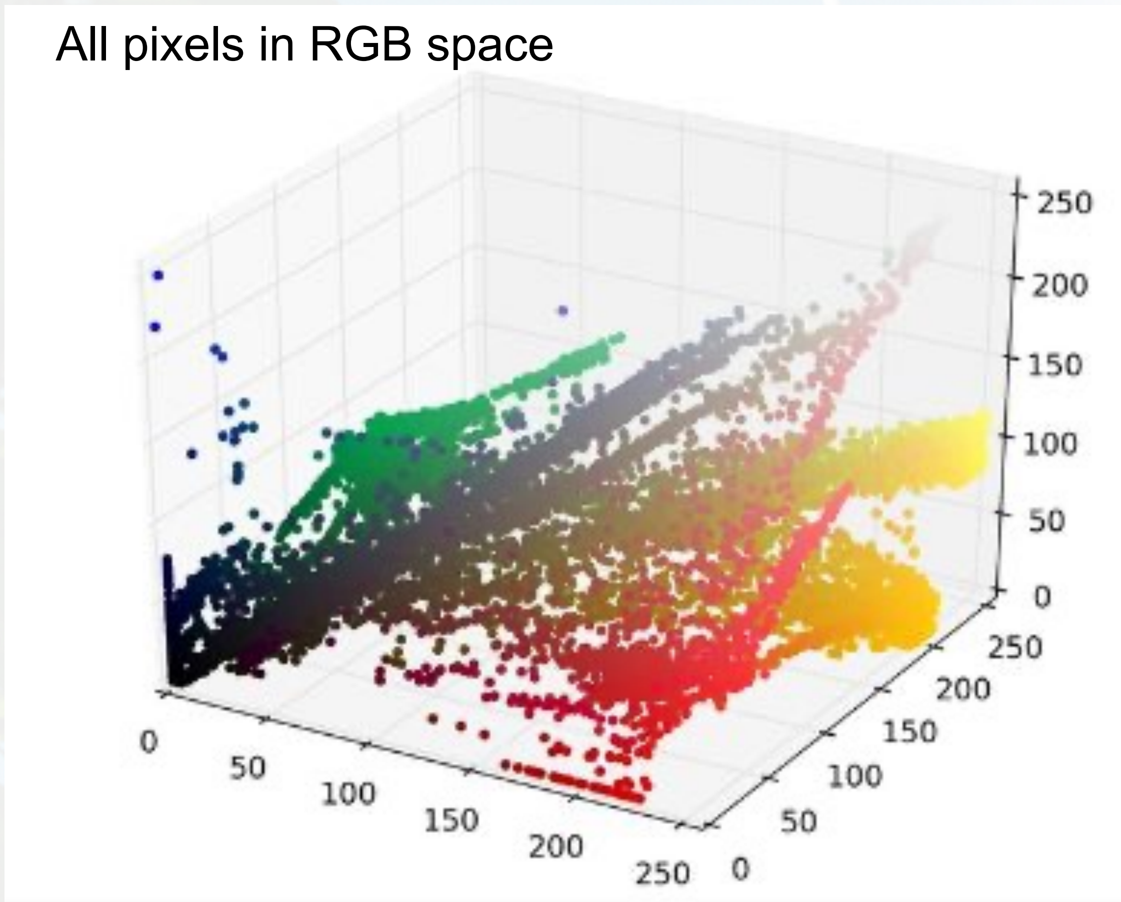
- Output depends on window size
- Window size (bandwidth) selection is not trivial
- Computationally (relatively) expensive (approx 2s/image)
- Doesn’t scale well with dimension of feature space.

Mean Shift Pros and Cons

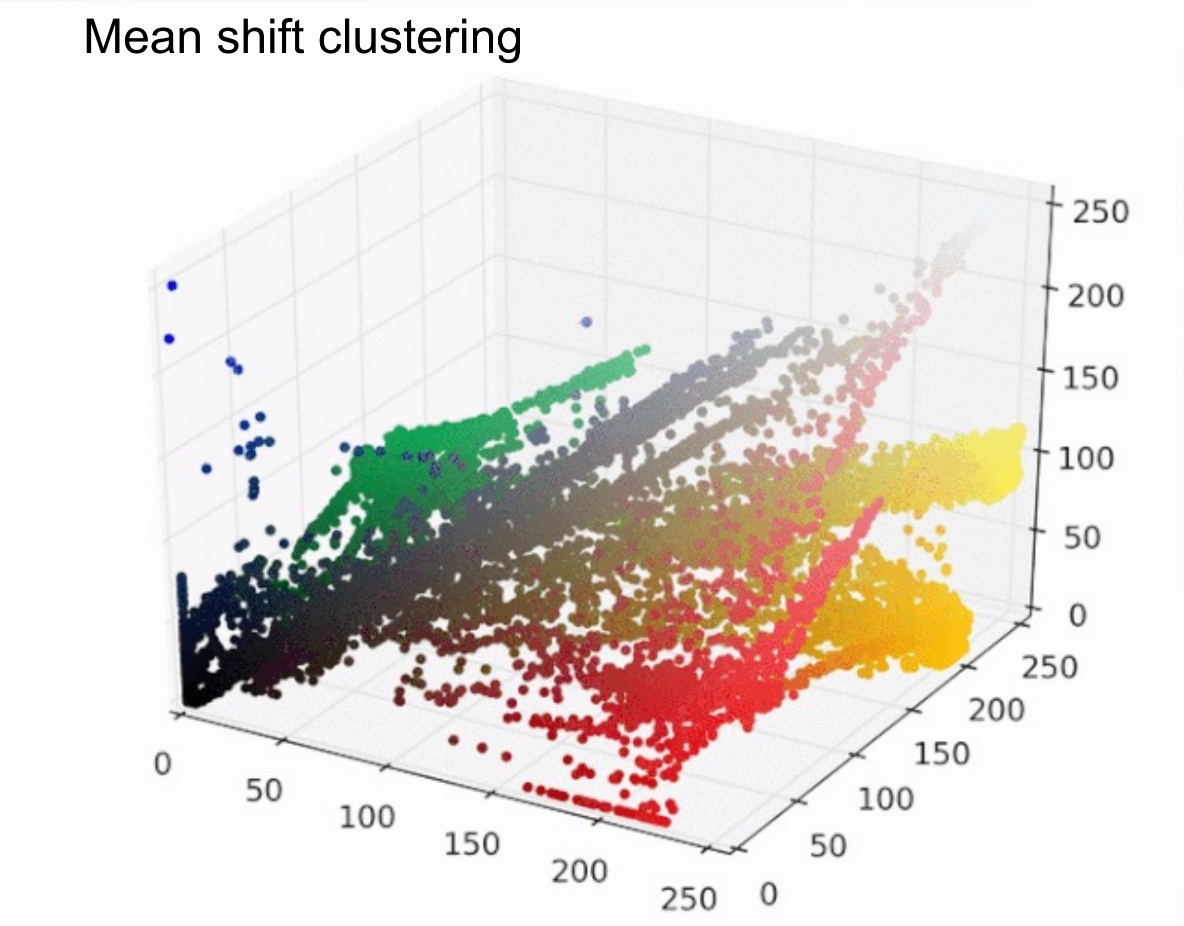


Mean Shift for Segmentation

Mean Shift for Segmentation



Mean Shift for Segmentation



Mean Shift for Segmentation